
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 15:06:11
Файлов исходного кода: 85
Суммарный объём кода: 1.15 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борушка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасика [id: aho-corasick]

Билеты (учебный контент)

- 24. src/data/tickets/ticket1_5.ts
- 25. src/data/tickets/ticket14_20.ts
- 26. src/data/tickets/ticket21_24.ts
- 27. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 28. src/components/ArticulationPointsViz.tsx
- 29. src/components/BellmanFordViz.tsx
- 30. src/components/BfsViz.tsx
- 31. src/components/ChapterImage.tsx
- 32. src/components/ChapterNav.tsx
- 33. src/components/ChapterView.tsx
- 34. src/components/DfsBridgesSimulator.tsx
- 35. src/components/DijkstraViz.tsx
- 36. src/components/DPVisualizer.tsx
- 37. src/components/EulerSimulator.tsx
- 38. src/components/ExpressQuiz.tsx
- 39. src/components/FloydViz.tsx
- 40. src/components/GraphTraversalViz.tsx
- 41. src/components/HeapViz.tsx
- 42. src/components/hints/demoKit.tsx
- 43. src/components/hints/demosExtra.tsx
- 44. src/components/hints/demosGraph.tsx
- 45. src/components/hints/demosStruct.tsx
- 46. src/components/hints/MiniDemo.tsx
- 47. src/components/hints/SimulatorModal.tsx
- 48. src/components/hints/TermPopover.tsx
- 49. src/components/JohnsonViz.tsx
- 50. src/components/KosarajuViz.tsx
- 51. src/components/KruskalSimulator.tsx
- 52. src/components/MnemonicCards.tsx
- 53. src/components/MobileToc.tsx
- 54. src/components/Navbar.tsx
- 55. src/components/PlanarityDemo.tsx
- 56. src/components/PythonCompiler.tsx
- 57. src/components/QueueViz.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/85: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

 deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
```

```
+   name: Publish to GitHub Pages
+   needs: build
+   runs-on: ubuntu-latest
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+   ....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
#
|
|→ page-0001.png ... page-NNNN.png (
#
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

```
На Ubuntu политика ImageMagick иногда запрещает экспорт
разрешаем то, что нужно пайплайну (text -> pdf)
for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml; do
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{PDF,PS\}" rights="write" pattern="\"/>
 fi
done
convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${GITHUB\_EVENT\_INPUTS.density:-300}  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages` |  
 echo "| \`\${GITHUB\_WORKSPACE}/public/export/full\_code\_all\_pages` |  
 echo "| PNG-страниц | \$(ls tmp/export-pages | wc -l) |"  
 } >> "\$GITHUB\_STEP\_SUMMARY"

```
+++ b/.github/workflows/rebuild-pdf.yml
```

```
@@ -42,15 +42,15 @@ jobs:
```

```
 steps:
```

```
 - name: Checkout
```

```
- uses: actions/checkout@v4
```

```
+ uses: actions/checkout@v6
```

```
 with:
```

```
 fetch-depth: 0
```

```
 persist-credentials: true
```

```
 - name: Setup Node.js
```

```
- uses: actions/setup-node@v4
```

```
+ uses: actions/setup-node@v7
```

```
 with:
```

```
- node-version: "22"
```

```
+ node-version: "24"
```

```
 cache: npm
```

```
 - name: Install ImageMagick + DejaVu fonts
```

```
@@ -97,7 +97,7 @@ jobs:
```

```
 } >> "$GITHUB_STEP_SUMMARY"
```

```
 - name: Upload artifacts (PDF + TXT)
```

```
- uses: actions/upload-artifact@v4
```

```
+ uses: actions/upload-artifact@v7
```

```
 with:
```

```
 name: full-code-export
```

```
 path: |
```

```
@@ -107,7 +107,7 @@ jobs:
```

```
 retention-days: 30
```

```
 - name: Upload artifacts (PNG-страницы)
```

```
- uses: actions/upload-artifact@v4
```

```
+ uses: actions/upload-artifact@v7
```

```
 with:
```

```
 name: full-code-pages-png
```

```
 path: tmp/export-pages/*.png
```

## `public/favicon.svg`

### Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
  <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

### Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
```

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
+  <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+  <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+  <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
```

````

## `src/App.tsx`

### Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate
```

```
    <Navbar activeTab={activeTab} setActiveTab={setAc
```

```
    {activeTab === "guide" ? (
```

```
      <>
```

```
        <MobileToc
```

```
          selectedChapterId={activeChapterId}
```

```
          setSelectedChapterId={goTo}
```

```
          currentTitle={activeChapter.title}
```

```
          index={index}
```

```
          total={chapters.length}
```

```
        />
```

```
      <div className="flex flex-col lg:flex-row max
```

```
        {/* Сайдбар только на десктопе – на мобильном
```

```
        <div className="hidden lg:block lg:w-80 shr
```

```
          <Sidebar selectedChapterId={activeChapter
```

```
        </div>
```

```
      <main id="main-content" className="flex-1 m
```

```
        {/* Шапка главы с быстрыми переходами */}
```

```
        <div className="flex items-center justify
```

```
          <div className="min-w-0">
```

```
            <div className="text-[10px] uppercase
```

```
              {activeChapter.category || "Универс
```

```
            </div>
```

```
            <h2 className="text-base sm:text-xl f
```

```
              {activeChapter.title}
```

```
            </h2>
```

```
          </div>
```

```
          <ChapterNav prev={prev} next={next} ind
```

```
        </div>
```

```
      <div className="h-1 w-full bg-slate-800 r
```

```
        <div
```

```
    });
  }
  ....
```

Patch относительно `main`

```
````diff
```

```
diff --git a/src/App.tsx b/src/App.tsx
```

```
index 2091f33..4583756 100644
```

```
--- a/src/App.tsx
```

```
+++ b/src/App.tsx
```

```
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
```

```
import { ChapterNav } from "../components/ChapterNav";
```

```
import { SimulatorModal } from "../components/hints/SimulatorModal";
```

```
import { SimulatorHub } from "../components/SimulatorHub";
```

```
+import { PythonCompiler } from "../components/PythonCompiler";
```

```
export default function App() {
```

```
- const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");
```

```
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator">("guide");
```

```
 const [activeChapterId, setActiveChapterId] = useState<string>("");
```

```
 const [modalVizId, setModalVizId] = useState<string>("");
```

```
@@ -95,7 +96,7 @@ export default function App() {
```

```
 </main>
```

```
 </div>
```

```
 </>
```

```
-) : (
```

```
+) : activeTab === "simulator" ? (
```

```
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
```

```
 <SimulatorHub
```

```
 onOpen={setModalVizId}
```

```
@@ -105,6 +106,8 @@ export default function App() {
```

```
 </main>
```

```
 </>
```

```
 </main>
```

```
+) : (
```

```
+ <PythonCompiler />
```

```

 k-0">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-extrabold">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
</div>

<div className="flex items-center w-full lg:w-auto">
 >
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-center justify-center
 duration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры

```

```
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </div>
</header>
);
};
},
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'

  interface NavbarProps {
-   activeTab: 'guide' | 'simulator';
-   setActiveTab: (tab: 'guide' | 'simulator') => void;
+   activeTab: 'guide' | 'simulator' | 'compiler';
+   setActiveTab: (tab: 'guide' | 'simulator' | 'compile
  }

  export const Navbar: React.FC<NavbarProps> = ({ active
```

```
-----
````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в браузер
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_INDEX_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) {
 return error.message;
 }
 return String(error);
}
```

```

 setRuntimeMessage("Не удалось выполнить код — проп
 } finally {
 setRunning(false);
 }
}, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]>
 <section className="rounded-2xl border border-slate-200 p-4">

```

```

 aria-label="Код Python"
 className="block w-full min-h-[360px] res
 -none focus:ring-2 focus:ring-inset focus:r
 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 b
 <label className="block px-4 pt-3 text-[1
 Ввод для input() • по одной строке на к
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.tar
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-tra
 eholder:text-slate-700"
 />
 </div>
</section>

<section className="rounded-2xl border border
 <div className="flex items-center justify-b
 <div className="flex items-center gap-2 t
 <Terminal className="w-4 h-4 text-emera
 </div>
 <span className={`inline-flex items-cente
 {runtimeReady && <CheckCircle2 classNam
 {runtimeReady ? "готов" : "не загружен"

 </div>
 <pre className="min-h-[360px] max-h-[560px]
 x] leading-6 text-slate-300">
 {output}
 </pre>
 <div className="border-t border-slate-800 p

```

```
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
```



```

+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу
+ });
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:i
+ <div>
+ <div className="inline-flex items-center g
+ <Terminal className="w-4 h-4" /> Боковая
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-e
+ <p className="text-slate-400 mt-2 max-w-2x
+ Пишите небольшой код и запускайте его пр
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5
+ >
+ Как это работает <ExternalLink className="
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1.
+ <section className="rounded-2xl border borde
+ <div className="flex flex-wrap items-cente
+ <div className="flex items-center gap-2"
+ <span className="w-2.5 h-2.5 rounded-f
+ <span className="text-sm font-bold tex
+ <span className="text-[11px] text-slate
+ </div>
+ <div className="flex items-center gap-2"

```

```

+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-l
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emerg
+ </div>
+ <span className={`inline-flex items-centr
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3
+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-ind
+ Ctrl или Cmd + Enter запускает код.

```

```

 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
\`\`\`

```

### Patch относительно `main`

```

\`\`\`diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
\`\`\`

```

3. **\*\*Строгая структура контента:\*\***

- Заголовок с цветным бейджем (например, `bg-indigo-100`)
- Определение/правило в центре (`border-blue-500` или `border-blue-500/2px`)
- Обязательная сетка аналогий (2 колонки: неформальный текст и код)
- "Душные" детали (код, формулы, сложные доказательства)

4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

---

ФАЙЛ 4/85: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600 Б

---

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
 <head>
 <meta charset="UTF-8" />
 <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
 <link rel="icon" type="image/svg+xml" href="%BASE_URL%/icon.svg" />
 <title> Дискретка для тупых – Интерактивный гид по дискретной математике
 </head>
 <body class="bg-slate-950 text-slate-100 min-h-screen">
 <div id="root"></div>
 <script type="module" src="/src/main.tsx"></script>
 </body>
</html>
```

ФАЙЛ 5/85: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 310 Б

---

```
{
 "name": "Remix Remix: ExamPrep Reader",
 "description": "Удобная web-читалка для подготовки к экзаменам по дискретной математике",
 "requestFramePermissions": [],
 "scripts": [
 {
 "src": "/src/main.tsx",
 "type": "module"
 }
]
}
```

## Универсальное пособие • полный исходный код

```

 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.28.2",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
 "description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/85: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР: 1000

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом  
и автоматическим экспортом всего исходного кода в PDF.

## Быстрый старт

```
```bash  
npm install  
npm run dev          # предпросмотр на http://0.0.0.0:5174  
npm run build        # сборка в dist/ (single-file)  
```
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в один файл

| Команда | Шаг | Результат |
|---------|-----|-----------|
|---------|-----|-----------|

## Универсальное пособие • полный исходный код

- на телефоне – липкая строка «Содержание · N из M» и в
- \* **\*\*Переходы между темами\*\*** – кнопки «Назад / Вперёд» в «Предыдущая / Следующая тема» под текстом; на клавиатуре
- \* **\*\*Всплывающие подсказки по терминам\*\*** – текст главы и известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор Словарь – `src/data/glossary.ts`, мини-анимации – `src/`
- \* **\*\*Реестр интерактивов\*\*** – `src/components/vizRegistry` и для панели под главой, и для модалки из подсказки.

### ## Структура

...

src/

- App.tsx – оболочка, привязка глав к визуализации
- components/ – интерактивные симуляторы (Действия)
- data/ – контент пособия (главы/билеты и т.д.)
- scripts/export/ – пайплайн код → txt → png → pdf
- public/export/ – сгенерированные TXT и PDF (обновление)
- .github/workflows/ – автоматизация

...

---

### # Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте теги и классы.

### ## Заголовки

- \* Заголовки секций: Используют стандартный тег `

## ` и
- \* Подзаголовки: Используют тег `

### `.

### ## Текст и параграфы

- \* Обычный текст содержится в тегах `

`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика: параграфы `

`.

Универсальное пособие • полный исходный код

>  
> Всего глав в пособии: **\*\*25\*\***.

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава              | id        | Что делать           |
|--------------------|-----------|----------------------|
| ---                | ---       | ---                  |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

## Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет — и

| Билет         | Тема                           | Статус                                 |
|---------------|--------------------------------|----------------------------------------|
| ---           | ---                            | ---                                    |
| <b>**1**</b>  | Дерево отрезков (Segment Tree) | Главы нет. Книжка читается вхолостую.  |
| <b>**7**</b>  | Планарность и раскраска графов | Номерной главы нет; аналогии**.        |
| <b>**11**</b> | Мосты в графах                 | Номерной главы нет; есть без аналогии. |
| <b>**13**</b> | Эйлеровы пути и циклы          | Номерной главы нет; есть без аналогии. |

Дополнительно — «осиротевшие» визуализаторы без глав (книжки)

- \* `stack-dfs` → `StackViz.tsx` — **\*\*Стек и DFS\*\***
- \* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **\*\*Очередь и BFS\*\***
- \* `heap-beam-search` → `HeapViz.tsx` — **\*\*Куча и лучевой поиск\*\***
- \* `segment-trees` → `SegmentTreeVisualizer.tsx` — **\*\*Дерево отрезков\*\***
- \* `dynamic-programming` → `DPVisualizer.tsx` — **\*\*Динамическое программирование\*\***

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

## Универсальное пособие • полный исходный код

|    |                                          |   |   |   |
|----|------------------------------------------|---|---|---|
| 12 | 12. Графы. Точки сочленения              | 1 | - | - |
| 14 | 14. Графы. Кратчайший путь. Дейкстра     | 3 | - | - |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман | 1 |   |   |
| 16 | 16. Графы. Кратчайший путь. Флойд        | - | - | - |
| 17 | 17. Графы. Алгоритм Джонсона             | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал      | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима        | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка      | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП             | 4 | - | - |
| 22 | 22. Z-функция                            | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик                 | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач     | 4 | - | - |
| -  | Обзор: Кратчайшие пути и Графы           | - | - | - |
| -  | Алгоритмы на карте                       | - | - | - |
| -  | Эйлер: Путь ≠ Цикл                       | - | - | - |
| -  | Мосты в графах: код + визуализация       | 1 | - | - |

## ## Рекомендуемый порядок работ

1. **\*\*Вернуть потерянные билеты 1, 7, 11, 13\*\*** и подключить ``segment-trees``, ``stack-dfs``, ``queue-bfs``, ``heap-be`` — это 5 готовых компонентов, которые сейчас просто не
2. **\*\*Дописать аналогии\*\*** в 5 главах из «Уровня 3» (по ш
3. **\*\*Добавить 2D\*\*** к 4 главам «Уровня 4» — минимум `inli`
4. **\*\*Решить судьбу `alg-map`\*\*** — раскрыть или удалить.

ФАЙЛ 9/85: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
```



```
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
```

```

 requestAnimationFrame(() => {
 requestAnimationFrame(() => {
 document.getElementById("chapter-viz)?.scrollIntoView();
 });
 });
 }, []));

// Стрелки ← → листают темы (как на обучающих сайтах)
useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target?.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index]);

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 activeTab={activeTab}
 setActiveTab={setActiveTab}
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

 <div className={compilerOpen ? "max-lg:pb-[58dvh] max-lg:pt-[58dvh] min-h-[58dvh] min-w-[58dvw] flex flex-direction: column-reverse" : ""}>
 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 />
 </>
) : (
 <MobileCodeEditor
 code={code}
 setCode={setCode}
 compilerOpen={compilerOpen}
 onToggleCompiler={onToggleCompiler}
 />
)}
 </div>
 </div>
);

```

```

 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>
) : (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8">
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
)}

 <footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное руководство
 </footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() => setModalVizId(null)}>
 {compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 onOpenGuide={() => setActiveTab("guide")}
 onClose={() => setCompilerOpen(false)}
 />
)}
</div>
);
}

```

```

*/
export function annotateTerms(root: HTMLElement): void {
 if (!cachedRegex) cachedRegex = buildRegex();
 const re = cachedRegex;

 const perTerm = new Map<string, number>();
 const perSurface = new Map<string, number>();
 const walker = document.createTreeWalker(root, NodeFilter.FILTER_ACCEPT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0) return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
);

 const targets: Text[] = [];
 let current = walker.nextNode();
 while (current) {
 targets.push(current as Text);
 current = walker.nextNode();
 }

 for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

```

```
 } catch {
 /* если браузер не умеет lookbehind – просто оста
 }
 };

 // при смене главы
 // eslint-disable-next-line react-hooks/exhaustive-dep
 useEffect(run, deps);

 // страховка: контент перерисовали, а подсказок в нём
 useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
 });
}
```

---

ФАЙЛ 13/85: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 КБ

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
}
```

```

 transform: translateY(4px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
}

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {

```

```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

-----  
}

-----  
ФАЙЛ 16/85: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В  
-----

```
/// <reference types="vite/client" />
```

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

=====

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

=====

-----  
ФАЙЛ 17/85: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 934 | РАЗМЕР: 10 КБ  
-----

```
import { Chapter } from "../types";
```



```

 <p class="text-slate-300 text-sm mb-4">
 нужно покрыть отрезок, мы берем две самые бо
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky)
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
 </div>
 </div>
</section>`,
 },
 {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```



каждой вставки чинят дерево, чтобы оно <em>  
</li>

<li><span class="text-white font-bold">  
менно быть кривым. Но каждый раз, когда мы  
ровнее становится.</li>

</ul>

</div>

<!-- 2. Аналогии -->

<div class="grid grid-cols-1 xl:grid-cols-2 gap-

<div class="bg-slate-800 p-6 rounded-lg bor-

<div class="absolute -top-3 left-4 bg-s-  
border-emerald-500 shadow-md">

Аналогия 1: стопка бумаг на столе

</div>

<p class="text-slate-300 text-sm">У теб  
скиваешь его и, поработав, кладёшь <span cl  
неделю самые нужные бумаги сами собой оказ  
ет ровно это: «взял → положил наверх».</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg bor-

<div class="absolute -top-3 left-4 bg-r  
der-rose-500 shadow-md">

Аналогия 2: тропинка через газон

</div>

<p class="text-slate-300 text-sm">Ты ид  
ода дорожка сама укорачивалась вдвое: чем ч  
оплачивает все следующие. Пойдёшь один раз

</div>

</div>

<!-- 3. Поворот -->

<div class="bg-slate-800/60 p-6 rounded-xl bord-

<h3 class="text-lg font-bold text-white mb-

<p class="text-slate-300 text-sm mb-4">Повор  
движение, три перевешенные ссылки</span> и

<pre class="bg-slate-950 p-4 rounded-lg ove  
ading-relaxed"> 40

```

 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">деда
 <td class="py-3 pr-3">деда
 <td class="py-3 pr-3">сам х
 <td class="py-3">х стал кор
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-3 pr-3 font-bolder">
аво-право)</td>
 <td class="py-3 pr-3">
 <td class="py-3">х поднялся
 </tr>
 <tr>
 <td class="py-3 pr-3 font-bolder">
 <td class="py-3 pr-3">х и р
 <td class="py-3 pr-3">
 <td class="py-3">р и г стал
 </tr>
 </tbody>
 </table>
 </div>

```

```

 <p class="text-slate-300 text-sm mt-4">И та
нова смотрим на тройку. Zig может случиться
 <p class="text-slate-400 text-xs mt-3"> Ни
вороты можно поделать руками.</p>

```

```
</div>
```

```
<!-- 5. Главная ловушка -->
```

```

<div class="bg-rose-950/30 p-6 rounded-xl borde
 <h3 class="text-lg font-bold text-rose-300 m
 <p class="text-slate-300 text-sm mb-4">Это
ми.</p>
 <div class="grid grid-cols-1 lg:grid-cols-2
 <div class="bg-slate-950 rounded-lg p-4
 <p class="text-rose-400 font-bold t

```

```
<pre class="bg-slate-950 p-4 rounded-lg over
0 leading-relaxed">старт после Zi
 глубина 3 → 1 10 в корне
```

```
 40
 /
 30
 /
20
/
10
```

```
 30
 / \
 10 40
 \
 20
```

```
 10
 \
 30
 / \
20 40
```

итог: было 4 узла в линию (высота 4) → стало высота 3,  
а сама десятка теперь в корне: следующий запрос к

```
<p class="text-slate-300 text-sm mt-4">Обра
инили дерево по пути. Все узлы, мимо
</div>
```

```
<!-- 7. Амортизация -->
```

```
<div class="bg-slate-800/60 p-6 rounded-xl bord
 <h3 class="text-lg font-bold text-white mb-1
 <p class="text-slate-300 text-sm mb-3">Стро
оддерева) по всем узлам, и Zig-Zig/Zig-Zag :
ах идея такая:</p>
 <div class="grid grid-cols-1 md:grid-cols-3
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">c
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">H
 </div>
 <div class="bg-slate-900 rounded-lg p-4
 <p class="text-white font-bold mb-1
 <p class="text-slate-400 text-xs">д
 </div>
 </div>
```

```

 const p = x.parent, g = p.parent;

 if (p.left === x) { // правый поворот
 p.left = x.right;
 if (x.right) x.right.parent = p;
 x.right = p;
 } else { // левый поворот (зеркало)
 p.right = x.left;
 if (x.left) x.left.parent = p;
 x.left = p;
 }

 p.parent = x; // родитель уехал вниз
 x.parent = g; // x занял его место
 if (g) { if (g.left === p) g.left = x; else g.right = x; }

// поднимаем x в корень
function splay(x) {
 while (x.parent) {
 const p = x.parent, g = p.parent;

 if (!g) { // ZIG: деда не осталось
 rotate(x);
 } else if ((g.left === p) === (p.left === x)) {
 rotate(p); // ZIG-ZIG: сносим
 rotate(x);
 } else {
 rotate(x); // ZIG-ZAG: сносим
 rotate(x);
 }
 }
 return x; // теперь x — корень
}

// поиск: спустились и подняли найденное наверх
function find(root, key) {
 let cur = root, last = null;

```

```

open: border-b border-slate-700/50">
 Вопросы, которые задают на экзамене (
</summary>
<div class="p-5 text-sm text-slate-300 space-y-4">
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Последний
 енный или преемник). Splay делают всегда, да-
 бы, и амортизация сломалась бы.</p>
 </div>
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Потому что
 еворачивают бамбук в другую сторону (см. бл-
 </div>
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Чередовани-
 вает один из них в корень, превращая дерево
 ступает только для одной конкретной
 </div>
 <div>
 <strong class="text-white block mb-2">Вопрос
 <p class="text-slate-400">Treap дер-
 play не хранит вообще ничего и держит балан-
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сжатие суммы подматрицы за O(1)",
 }
}

```

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ый граф можно раскрасить 4 цветами.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```



```

 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
нвертированному в порядке top-sort.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением.
 </div>
 <p class="text-slate-300 text-sm mb-4">
 В некоторых городах существуют односторонние улицы, что может повлиять на направление движения. Проверьте направление движения на карте.</p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает граф",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-rose-400
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-rose-300 italic
 <p class="text-xl font-mono text-white"
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb
правильный роутер - и два сегмента офиса б
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-40
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-indigo-300 italic

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ
 </div>
</div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border" style="border: 1px solid #008000; border-radius: 10px; box-shadow: 5px 5px 0px #008000;">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 (проверяем через DSU) - берем в ответ.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">

```

```
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
</div>
</div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия: Племена
 </div>
 <p class="text-slate-300 text-sm mb-4">
 изкому племени для объединения. К вечеру пле
 ства шлют гонцов к ближайшему чужому царству.
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
}
```

```

 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-blue-300 italic
 <p class="text-xl font-mono text-white`
```

```

content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-400
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-purple-300 italic
 <p class="text-xl font-mono text-white"
Сведение (Reduction) A->B: Если я умею реша
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Судoku (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb
пример сортировка чисел). Класс NP –
енную сетку (ответ), он БЫСТРО (за класс P)
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-purple-500 shadow-md">
 Аналогия 2: Машина-переводчик
 </div>
 <p class="text-slate-300 text-sm mb
ь отличный переводчик на английский (Сведен
аче B, то B – это NP-Полная (сосредо
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
`

```

```

 <div class="p-3 bg-slate-950 rounded
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

* <span class="text-green-400 font-
шел-вошёл-вышел).

* <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Путь: Нормальная прогулка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md bg-slate-950">
 Цикл: Гамильтонов синдром (болезнь
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Гамильтон – это болезнь.</s
ng> ровно раз и вернуться домой.

```

```
<h2 class="text-3xl font-bold text-white">Мосты
</div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-400
 </div>
 </div>
 </div>
 </div>
 </div>
```

```
<p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков
 Единственная ниточка. Порвётся – граф p
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap">
 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-900
 border-emerald-500 shadow-md">
```

Мост: Единственная веревка

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Представь, что ты альпинист. Ты спу
 Из пещеры и нет других выходов – то
 Если верёвка (ребро v-u) оборвётся -
 Если бы из пещеры и был чёрный ход
</p>
```



```

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

```

```

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);

 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
}

```

</div>

<p class="text-xs text-slate-400">Запуск

</div>

</details>

</div>

```

 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
 -white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 – эталон не
 Если внутри графа спрятан K5 – забудь,
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6,
 xt-white">9 ≤ 12</code> – проходит, но он не
 Почему? Потому что у двудольного графа
 Отсюда более строгое ограничение: <code class="text-white">9 > 8</code>
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-rose-500 shadow-md">
 <summary class="p-4 font-bold text-slate-400">
 Доказательство непланарности K5 (Скрыть)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400">Доказательство
 <p>
 Пусть K5 планарен. $V = 5, E = 10$.

 $F = E - V + 2 = 10 - 5 + 2 = 7$

Каждая грань ограничена минимум

```

еются к точкам сочленения. То есть, **ег** - это тупик со степенью 1).

</p>

</div>

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-900">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

</div>

<p class="text-slate-300 text-sm mb-4">

Представь город, где два района соединены перекрестком. Если один из районов будет **хрупкостью** системы.

</p>

</div>

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-950">
 <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной сети. Если один из них выйдет из строя, это может быть **абелем** (мостом). Сами свитчи — это точки сочленения.

</p>

</div>

</div>

```
<details class="bg-slate-900/40 rounded-lg border border-slate-950">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
```

</summary>

<div class="p-5 text-sm text-slate-300 space-y-2">

<p class="text-blue-400 font-bold">Особые случаи

<p>

```
 <strong class="text-emerald-400">SCC-зависимые "конгломераты".
 Как точка сочленения рушит Косарайю, свернув каждую SCC в одну супер-вершину (сделаем неориентированным), то любая сильная компонента и есть критическая SCC! Если удалить эту вершину. Одно слабое звено изолирует целые касты SCC!

</div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 19/85: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 51 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as newChapters } from "../content_temp";

// We want to combine all of them, but overwrite 7, 11, 12, 13
const merged = [
 ...graphChapters,
 ...additionalTickets,
 ...tickets6to13,
 ...tickets14to20,
 ...tickets21to24
].filter(c => c && c.id);

// Remove old 7, 11, 12, 13: graph-planar-colors, graph-planar-colors, graph-planar-colors, graph-planar-colors
```

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

```
 "Волна от источника: сначала все соседи, потом со
```

```
 formal:
```

```
 "Очередь (FIFO). Кладём старт, достаём вершину –
```

```
 у рёбер.",
```

```
 complexity: "O(V + E)",
```

```
 demo: "bfs",
```

```

 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот п
 formal: "push_back / pop_front за $O(1)$. Основа BFS
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "СНМ", "система непересекающихся мн
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего
 formal:
 "find со сжатием путей + union по рангу/размеру.
 complexity: "почти $O(1)$ — обратная функция Аккерман
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "t
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово
 formal: "Из корня по символам; в вершине хранится ф
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
 demo: "trie",
 simulator: "aho-corasick",
},
{
 id: "bfs-on-trie",
 labels: [
 "обход дерева",
 "обход бора",
 "обход в ширину по бору",
 "какой-то обход",
 "обходом дерева",
 "конечный автомат",
],

```

```

{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if $d[v] > d[u] + w(u,v) \rightarrow d[v] = d[u] + w(u,v)$
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "префикс"],
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от начала»
 formal: " $P[0]=0, P[i]=P[i-1]+a[i-1]$. Сумма $a[l..r]$ = $P[r]-P[l-1]$.",
 complexity: "препроцессинг $O(n)$, запрос $O(1)$ ",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [
 "разреженная таблица", "разреженная таблица", "sparse-table",
 "разреженная", "двоичные подьёмы", "двоичный подьём",
],
 title: "Разреженная таблица (метод двоичных подьёмов)",
 short:
 "Заранее считаем ответ для всех отрезков длины 1, 2, 4, 8, ...,
 м.",
 formal: " $ST[i][j] = op(ST[i][j-1], ST[i+2^{(j-1)}][j-1])$ ",
 complexity: "препроцессинг $O(n \log n)$, запрос $O(1)$ ",
 demo: "sparse-table",
 simulator: "sparse-table",
},

```

```

],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная
 formal: "V-1 ребро, никаких циклов. Краскал (жадно
 но).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи
 "амортизированное", "амортизированного", "амортизи
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер
 formal: "Метод потенциалов: дорогая операция оплачи
 ассив.",
 demo: "amortized",
},
{
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн
 "класс P", "полином", "полиномиальное",
],
 title: "Класс NP и NP-полнота",
 short:
 "NP — «решение проверить легко, найти трудно» (су
 ые в NP.",
 formal: "Задача NP-полна, если она в NP и к ней полн
 demo: "np",
},
{
 id: "potential",
 labels: ["потенциал", "потенциалы", "потенциалов",

```



```

 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "Z-функция"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начинающихся с этой буквы.",
 formal:
 "z[i] = длина наибольшего общего префикса s и s[i..n]",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "динамическое",
 "мемоизация", "мемоизацию", "подзадач", "подзадачи",
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и подзадачи.",
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся подзадачи.",
 demo: "dp",
 simulator: "dynamic-programming",
},
{
 id: "shortest-path",
 labels: ["кратчайший путь", "кратчайшие пути", "кратчайшие"],
 title: "Кратчайший путь",
 short:
 "Самый дешёвый маршрут из A в B. «Дешёвый» — по стоимости.",
 formal:
 "Дейкстра — неотрицательные веса, O(E log V). Фор — отрицательные веса, O(V^3).",
 demo: "relax",
 simulator: "dijkstra",
},

```



```

 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажды
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prim",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
 },
 {
 id: "kruskal",
 labels: ["Краскал", "Краскала", "Краскалу", "Kruskal"],
 title: "Алгоритм Краскала",
 short:
 "Сортируем все рёбра по весу и берём подряд те, ч
 formal:
 "Сортировка O(E log E) + E операций union/find. С
 complexity: "O(E log E)",
 demo: "mst",
 simulator: "mst-kruskal",
 },
],

```

```

 "sort + unique + binary search: значение → индекс",
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова де",
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по случайности. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева через них. Ожидаемая высота O(log n).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Поиск по ключу.",
 formal:
 "Без балансировки дерево может вырождаться в список.",
 complexity: "O(высоты): от O(log n) до O(n)",
 demo: "segment-tree",
},
{
 id: "inclusion-exclusion",
 labels: [
 "включений-исключений", "включения-исключения", "принцип",
 "вырезание прямоугольников",
],
 title: "Включения-исключения на префиксных суммах",
 short:
 "Чтобы получить сумму прямоугольника, берём больше",

```

```

 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функции.
demo: "prefix-function",
simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо",
title: "Поворот (rotation) в дереве",
short:
 "Локальная перевеска двух узлов: ребёнок встаёт на
 дившемся месту. Порядок ключей при этом не лом
formal:
 "Правый поворот вокруг p: $x = p.left$; $p.left = x$.
 g и Zig-Zag.",
complexity: "O(1)",
demo: "zig",
simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig — одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в кор
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию
demo: "zig",
simulator: "splay-rotations",
},
{
 id: "zig-zig",
 labels: ["Zig-Zig", "зиг-зиг", "ZigZig"],
 title: "Zig-Zig — x и родитель с одной стороны",
 short:
 "Дерево вытянулось в «бамбук»: $g \rightarrow p \rightarrow x$ идут в о
 м (p, x). Именно это вдвое сокращает глубину вс
 formal:
 "Если крутить наивно (два раза поднимать x), бамб

```

```

content: `<section id="intro-content" class="mb-12
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
},
{
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Дейкс
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <p class="text-xl font-mono text-white":
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2

```

```
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class=
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Форд-Беллман
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400">Аналогия 1: Параноик

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Параноик убежден, что все вокруг него заговорили
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 order border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">Параноик проверяет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman-ford" class="w-full h-40">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 border-rose-500 shadow-md">
 ♂ Отрицательный цикл
 </div>
 </div>
 </div>
 </div>
`
```

```

 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd"
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb
три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
</div>

 <div id="slot-floyd-viz" class="mt-8"></div>
</div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 roun
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0cr
 <p class="text-lg font-mono text-white">Бор (Tr

```



```

 <div class="bg-black/50 p-4 rounded-lg font-monospace">
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по суффиксу
 int term_link = 0; // Терминальная ссылка: мат. слово закончилось
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные link
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход по суффиксу родителя
 nodes[child].link = nodes[nodes[v].link].go[ch];

 // Если суффиксная ссылка сама является словом
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
 } else {
 nodes[child].term_link = nodes[nodes[child].link].term_link;
 }

 q.push(child);
 }
 }
}

```

```

 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётными степенями",
 options: [
 "Есть эйлеров цикл (вернусь домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если больше 2 — эйлеров цикла нет, и пути тоже нет, если нечётных вершин больше 2."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней. Это NP-полная задача, которую не так просто решить."
 },
 {
 question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
 options: [
 "Всё ещё можно пройти по всем рёбрам ровно раз",
 "Такого графа не существует",
 "Эйлерова пути нет — нужно удалять лишние рёбра"
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если нечётных вершин ровно 2 нечётные или 0."
 }
]

```

```

 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани про
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с $V=7$?",
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K_5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K_5 : $V=5$, $E=10$, $3V - 6 = 9$. $10 > 9$
 }
]
};

```

```

 { name: "l, r", role: "границы отрезка, за который",
 { name: "m", role: "середина отрезка $m = (l + r)$ " },
],
 },
 treap: {
 // у страницы нет интерактивного виджета
 variables: [],
 },
 "splay-tree": {
 vizTitle: "Splay-дерево (Zig / Zig-Zig / Zig-Zag)",
 stepNote: "Шаг = один поворот, поднимающий узел x на",
 variables: [
 { name: "x", role: "узел, который поднимаем к корню",
 { name: "p", role: "родитель узла x перед поворотом",
 { name: "g", role: "дедушка узла x – нужен в случае",
 { name: "zig / zag", role: "направление поворота:" },
],
 },
 "splay-rotations": {
 vizTitle: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
 stepNote: "Шаг = один кадр поворота из трёх классиче",
 variables: [
 { name: "x", role: "узел, который поднимаем к корню",
 { name: "p", role: "родитель узла x перед поворотом",
 { name: "g", role: "дедушка узла x – нужен в случае",
],
 },
 "sparse-table": {
 vizTitle: "Разреженная таблица (1D и 2D)",
 stepNote: "Шаг построения = заполнение ячейки уровня",
 variables: [
 { name: "n", role: "длина массива / сторона квадр",
 { name: "k", role: "уровень таблицы: ячейка храни",
 { name: "i", role: "начало блока в строке уровня",
 { name: "j", role: "номер уровня при построении (",
 { name: "r, c", role: "строка и столбец ячейки в ",
 { name: "kx, ky", role: "уровни по вертикали и гор" },
],
 },
 },
 },
 },
],
 },
],
 },
],
 },

```

```

],
 },
 "queue-bfs": {
 vizTitle: "Очередь и обход в ширину",
 stepNote: "Шаг = dequeue вершины из головы очереди",
 variables: [
 { name: "front", role: "голова очереди – вершина", range: 1, type: "int" },
 { name: "v", role: "вершина, извлечённая из очереди", range: 1, type: "int" },
 { name: "u", role: "сосед вершины v, которого кладем в очередь", range: 1, type: "int" },
],
 },
 "everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на стол / со стола",
 variables: [
 { name: "top / front", role: "верх стопки (стек) / начало очереди", range: 1, type: "int" },
 { name: "i", role: "индекс в массиве кучи; родитель вершины v", range: 1, type: "int" },
],
 },
 "graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к непосещённой",
 variables: [
 { name: "v", role: "текущая вершина обхода", range: 1, type: "int" },
 { name: "to / u", role: "очередной сосед вершины v", range: 1, type: "int" },
 { name: "visited", role: "множество уже посещённых вершин", range: 1, type: "bool" },
 { name: "order", role: "порядок, в котором обход графа", range: 1, type: "int" },
],
 },
 "graph-components": {
 // отдельного виджета нет, тема опирается на DFS/BFS
 variables: [],
 },
 "top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она добавляется в очередь",
 variables: [
 { name: "v", role: "текущая вершина DFS", range: 1, type: "int" },
],
 },

```

```

"euler-path-vs-cycle": {
 vizTitle: "Эйлеров путь и цикл",
 stepNote: "Шаг = один клик по следующей вершине маршрута",
 variables: [
 { name: "path", role: "текущий маршрут – последовательность вершин", range: "1..n", type: "text" },
 { name: "last", role: "последняя вершина пути – обрывок", range: "1..n", type: "text" },
 { name: "deg(v)", role: "степень вершины: чётная или нечётная", range: "1..n", type: "text" }
],
},
"planarity-euler-formula": {
 vizTitle: "Планарность: K5 и K3,3",
 stepNote: "Шаг = попытка перетащить вершину так, чтобы не было пересечений",
 variables: [
 { name: "V", role: "число вершин графа", range: "1..10", type: "text" },
 { name: "E", role: "число рёбер", range: "K5: 10, K3,3: 9", type: "text" },
 { name: "F", role: "число граней в плоской укладке", range: "1..10", type: "text" }
],
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = выбор ближайшей непосещённой вершины",
 variables: [
 { name: "u", role: "вершина с минимальным dist среди непосещённых", range: "1..n", type: "text" },
 { name: "v", role: "сосед вершины u, до которого можно добраться", range: "1..n", type: "text" },
 { name: "w", role: "вес ребра (u, v)", range: "по умолчанию 1", type: "text" },
 { name: "dist[v]", role: "лучшее известное расстояние от s до v", range: "1..n", type: "text" }
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в направлении u → v",
 variables: [
 { name: "n", role: "число вершин графа – ровно n-1", range: "1..10", type: "text" },
 { name: "m", role: "число рёбер, по которым проходятся", range: "1..10", type: "text" },
 { name: "i", role: "номер итерации (после i итераций все расстояния минимальны)", range: "1..m", type: "text" },
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес", range: "1..n", type: "text" },
 { name: "dist[v]", role: "расстояние; если обновлено, то выделено", range: "1..n", type: "text" }
],
},

```

```

 variables: [
 { name: "v", role: "вершина, которую только что д"},
 { name: "u", role: "ещё не взятый сосед v — канди"},
 { name: "w", role: "вес ребра (v, u) — ключ в куч"},
 { name: "start", role: "стартовая вершина, из неё"},
],
 },
 "mst-boruvka": {
 vizTitle: "Борувка",
 stepNote: "Шаг = каждая компонента одновременно выб",
 variables: [
 { name: "comp", role: "компонента связности текущ"},
 { name: "cheapest[comp]", role: "самое дешёвое ре"},
 { name: "u, v, w", role: "ребро-кандидат: концы и"},
],
 },
 "string-kmp": {
 vizTitle: "Префикс-функция (КМП)",
 stepNote: "Шаг = обработка символа s[i]: откаты по",
 variables: [
 { name: "n", role: "длина строки s, для которой с"},
 { name: "i", role: "индекс текущего символа — счи"},
 { name: "j", role: "длина текущего наилучшего сов"},
 { name: "π[i]", role: "префикс-функция: длина наи"},
 ица" },
],
 },
 "string-z-func": {
 vizTitle: "Z-функция",
 stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l",
 variables: [
 { name: "n", role: "длина строки s", range: "n ="},
 { name: "i", role: "позиция, для которой считаем"},
 { name: "l, r", role: "правый крайний Z-блок: отпр"},
 { name: "z[i]", role: "длина наибольшего общего пр"},
],
 },
 "aho-corasick": {

```

```
id: "segment-trees",
title: "1. Дерево отрезков с операциями на отрезках",
type: "html",
description: "Дерево отрезков – структура данных для...",
category: "Оптимизации и Продвинутое",
content: `
<section id="segment-trees" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

 <div class="space-y-8">
 <div id="intro" class="bg-slate-700/50 p-6 rounded">
 <h3 class="text-xl font-bold text-blue-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
omise (lazy).</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Корпоративная пре
 </div>
 <p class="text-slate-300 text-sm mb">
ы рассылать 10 000 писем, он пишет одно пис
счетах сотрудников только тогда, когда сотр
женное обновление).</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
 Аналогия 2: Матрешки-коробки
```



```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ух отрезков: $\min(ST[L][k], ST[R - 2^k + 1][k])$
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb-4">
 зок длины 7. Ты берешь две заготовленные заготовки
 ь отрезок длины 7 (перекрывая друг друга пополам)
 ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ужно покрыть отрезок, мы берем две самые большие
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">

```

```

<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #f4a460; border-radius: 10px; box-shadow: 0 10px 15px #f4a460; position: relative;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md" style="border: 1px solid #f4a460; border-radius: 10px; box-shadow: 0 10px 15px #f4a460;">
```

```

 <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300" style="background-color: #f0f0f0; border: 1px solid #ccc; border-radius: 10px; padding: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="background-color: #1a202c; color: #e2e3e5; border-radius: 10px; padding: 10px;">
// Zig, Zag
function rotate(x) {
 let p = x.parent;
 if (p.left === x) { // Right rotate
 p.left = x.right;
 x.right = p;
 } else { // Left rotate
 p.right = x.left;
 x.left = p;
 }
}
}</pre>
 </div>
 </details>

 <div class="bg-indigo-950/60 p-6 rounded-xl" style="background-color: #2d3748; color: #e2e3e5; border-radius: 10px; padding: 10px;">
 <h4 class="text-lg font-bold text-indigo-300">Сбалансированное дерево AVL</h4>

 <div class="space-y-4 text-sm text-slate-300">
 <div>
 <strong class="text-white block">Корень?
 <p>Оно поднимет узел, на котором произошла ротация, и сделает его корнем. За счет сдвига этого листа в корень, дерево становится сбалансированным.</p>
 </div>

 <div>
 <strong class="text-white block">Сбалансированное дерево?
 <p>Если агент постоянно переключается между двумя деревьями, находящимися на разных концах дерева, его можно представить как вытянутый бамбук для другого. Постоянное свинг-переключение превратит процесс в непрерывный.</p>
 </div>
 </div>
 </div>
 </div>
</pre>
```

```
 rder border-emerald-500 shadow-md">
 * Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb
з него маленький целевой прямоугольник в це
ый верхний угол отрезается дважды! Поэтому
 </div>
 </div>
</div>
</section>`
 }
};
```

---

ФАЙЛ 25/85: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
 {
```

```
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4">
 опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
</section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">

```

```

 id: "johnson-algo",
 title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-amber-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-amber-500 shadow-md">
 ⚙ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Дейкстра (Билет 14) быстрая, но работает очень медленно. Мы хотим сделать Джонсона пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 space-y-4">
 Добавляем фиктивную вершину
 Запускаем от неё медленно «потенциалы» <code class="text-pink-400">
 Меняем старые веса: новое расстояние = старое + потенциал
 Магия! Все веса теперь минимальные (потенциалы сокращаются).
 Теперь смело запускаем быстро

 </div>
 </div>
 </div>
 </div>
`

```

```

 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 19
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">Остовное дерево
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Остовное дерево
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Остовное дерево — это подграф, который
 <p class="text-xl font-mono text-white">содержит все вершины графа, но не содержит ребер,
 которое торчит из посещенных в непосещенные.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">Представим себе граф (или сеть), в котором нет ни одного
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
 </div>
</section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",

```

```
import { Chapter } from '../..types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Префикс-функция (π) – Взгляд назад (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Мы идём по строке слева направо.
 строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слово
 х</code>. Тупой алгоритм начнет искать за но
 </code>, а это начало нашего слова! Не надо
 </p>
 </div>
 </div>
 </div>
 </div>
`
 }
];
```



```

 </details>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {
 id: "string-z-func",
 title: "22. Z-функция – Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">

```

```

 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2">
 <code>Z[0]</code> – не счита
 <code>Z[1]</code> (bcabcd
[1] = 0</code>
 <code>Z[2]</code> (cabcd

 <code>Z[3]</code> (abcd<
? Нет, в начале 'a', тут 'd'). Совпало 3 бы
 <code>Z[4], Z[5], Z[6]</code>

 </div>
</details>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код C++ (Скрыто)
</summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `

```



ФАЙЛ 27/85: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 6
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">Графы
 </div>
 <div class="space-y-8">
 { /* База представлений */ }
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-slate-300">Представление графа
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">Граф — это набор вершин
 <p class="text-xl font-mono text-white">V
 (V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" м
 </div>
 </div>
 </div>
 </div>
 </div>
`
 }
];
```

```

<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 BFS (Поиск в ширину) = Волна
</div>
<p class="text-slate-300 text-sm mb">
 Что делает: Концентричес
 Работает через Очередь (FIFO). Продвигается
</p>
<ul class="text-xs text-emerald-300">
 Где используется:
 Кратчайший путь в Невзвешен
 Алгоритм Ахо-Корасик (сборка
 Алгоритм Форда-Фалкерсона /
 Двунаправленный поиск для у

</div>
</div>

<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300 border border-slate-700/50">
 Код обходов C++ (Основа)
 </summary>
 <div class="grid grid-cols-1 md:grid-col">
 <pre class="bg-slate-950 p-3 rounded">
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>> adj;

void dfs(int v) {
 vis[v] = true;
 for (int u : adj[v]) {
 if (!vis[u]) dfs(u);
 }
}</pre>

 <pre class="bg-slate-950 p-3 rounded">
// BFS (Очередь)
queue<int> q;

```

```

<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-6">
 . Границы не пересекаются в одной точке по-
 ы не сливались цветами, Всегда можно всего
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-1">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ество компонент.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l

```

```

 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 >
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выяв
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300 border border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-indigo-400 font-bold">
 <div class="bg-slate-950 p-4 rounded">
 <pre class="text-xs font-mono">
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological sort(int n) {

```

**«Это просто цикл?» – Нет!**

- Если у тебя есть цикл `A → B → A`
- Если ты добавишь вершину **Г**, получится четыре вершины – одна большая SCC.
- Правило: **Если два цикла имеют хотя бы одну общую вершину, то они принадлежат одной SCC.**

Аналогия: Интриги в офисе

Сильная связность – это "клуб" сотрудников, которые все в одной SCC. Добавить курьера Гошу, который может идти наружу (к директору), но обратно в клуб не может.

Связь с Билетом 12

**SCC (Билет 10)** склеивает вершины в одну компоненту.

**Точки сочленения (Билет 12)** – это вершины, удаление которых разбивает граф на несколько компонент.



```

 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb
 ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутое",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 r

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic r
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
 </div>
 <p>рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounded

```

```

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50"
 <summary class="p-4 font-bold text-slate-400"
 order-slate-700/50">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2"
 <p class="text-blue-400 font-bold">Особый
 <p>
 Для стартовой вершины обхода правил
 то выше неё прыгать некуда). Поэтому для неё
 у него больше 1 независимого ребенка
 </p>
 <div class="bg-slate-950 p-4 rounded-lg"
 <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
</pre>
 </div>
 </div>
 </details>

```

```
 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрыв
 </div>
 <p class="text-slate-300 text-sm mb">
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-выш
 </div>
 </div>
 </div>
 </div>
 </section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

---

ФАЙЛ 28/85: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-re

interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```

 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

```

```

 } else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
 }
 };

 for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
 }

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

<div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l
 <svg className="w-full h-full" viewBox="0 0 8
 { /* Edges */ }
 { edges.map((edge, i) => {
 const u = nodes.find((n) => n.id === edge
 const v = nodes.find((n) => n.id === edge

 const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

 return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : ""}
 />
);
 })
 }

 { /* Nodes */ }
 { nodes.map((node) => {
 const isCurrent = currentStep.currentNode
 const isVisited = currentStep.visited.has
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited

```

```

 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 tin:{" "}
 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
)}
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
```





```

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
 const list: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (list[n.id] = []));
 ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
 return list;
};
const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start)' },
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0;' },
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations' },
 { line: 4, text: ' for (u, v, weight) in E:' },
 { line: 5, text: ' if dist[u] + weight < dist[v]:' },
 { line: 6, text: ' dist[v] = dist[u] + weight;' },
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл' },
 { line: 8, text: ' if dist[u] + weight < dist[v]:' },
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
 { line: 10, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 99 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: 'Фура заведена. Начинаем с Базы (S). Расстояние до А: 99',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {

```

```

 simulationSteps.push({
 iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: ` ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}
 activeCodeLine: 9,
 });
 break;
 }
 }

 if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ` Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
 }
 }

 setSteps(simulationSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

```

```

 {currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-r
 animate-pulse shadow-lg shadow-rose-600/50
 ▲ Отрицательный цикл!
 </div>
)}

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth=
 path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u)
 const vNode = nodes.find((n) => n.id === v)
 const isChecking = currentStep.checkingEdges
 v;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D'
 && edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'
 else if (currentStep.hasNegativeCycle &&

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y}
 asNegativeCycle && isCyclePart) ? 4 : 2} ma
 g ? '4,4' : 'none' />
 <circle cx={(uNode.x + vNode.x) / 2}

```

```

 -colors">
 <span className="font-bold text-white
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingE
 return (
 <span key={idx} className={`px-2
edge.w < 0 ? 'bg-rose-900/80 text-rose-300
 {edge.v} ({edge.w})

)
 })}
 </div>
)})
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10
 <div>
 <h4 className="text-lg font-bold text-rose-:
 <div className="grid grid-cols-4 gap-2 text
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge
 return (
 <div key={n.id} className={`p-2 round
 : 'border-slate-700/60 bg-slate-800/40'}`}>
 <div className="text-xs text-slate
 <div className={`font-mono font-bo
 v>
 </div>
);
 })}
 </div>
 </div>
 </div>
</div>

```

```
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."
```

```
 });

 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
```

```

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
 });
 }
 } else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
 }
 }
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

```



```

 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-slate-400"
 >
 {n.label}
 </text>
 </g>
);
 }}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4 font-mono font-medium">
 bfs(graph, start)
 var {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visited.add(start);' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmpty() {',
 activeLine: 4 },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node] {',
 activeLine: 6 },
 { line: 7, code: ' if neighbor not in visited {',
 activeLine: 7 },
 { line: 8, code: ' visited.add(neighbor); queue.push(neighbor);',
 activeLine: 8 },
 { line: 9, code: ' }',
 activeLine: 9 },
 { line: 10, code: ' }',
 activeLine: 10 },
 { line: 11, code: '}' },
]}.map((cl) => {
 const isActive = step.activeLine === cl.line;
 return (
 <div key={cl.line} className={`px-2 py-1 ${
 isActive ? 'bg-slate-700 text-white' : 'bg-slate-800 text-slate-400'
 }`}

```



ФАЙЛ 31/85: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: 'Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
```



```

 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлен код */
 onOpenCompiler?: () => void;
}

const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;

export const ChapterView: React.FC<Props> = ({ chapter, ...props }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");

 useTermHints(contentRef, [chapter.id]);

 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, []);

 const scheduleHide = useCallback(() => {
 cancelHide();
 hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
 }, [cancelHide]);

 useEffect(() => {
 setAnchor(null);
 setSaving("idle");
 }, [chapter.id]);

```

```

 else open(el);
 };

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small border-1px
 border-indigo-500 px-3 py-2 300 hover:text-white hover:border-emerald-500"
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
 </button>
 <button
 onClick={() => window.print()}
 title="Распечатать или сохранить в PDF средствами браузера"
 className="flex items-center gap-1.5 text-x-small border-1px
 border-indigo-500 px-3 py-2 300 hover:text-white hover:border-indigo-500"
 >
 <Printer className="w-3.5 h-3.5" /> Печать
 </button>
 </div>
 </article>
 </>
);

```

```

 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python-компилятор: о
омментарии"
 className="flex items-center gap-1.5
t-emerald-400 hover:text-white hover:border
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
 </div>
 {viz.hint && <p className="text-xs text-sla
 <viz.Component />
 </section>
}}

 <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
/>
</>
);
};

```

```

 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}

const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {

```

```

 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:4,3:5,4:6},
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
 activeCodeLine: [11,13,16,17]
},

```

```

 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 }
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer);
 }, [dfsAuto]);

 const step = DFS_STEPS[dfsIdx];

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">Шаг {step}</h4>
 <div className="flex items-center gap-1 bg-slate-800 p-1">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 disabled={dfsIdx === 0}
 className="p-1.5 hover:bg-slate-700 rounded"
 >
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${
 dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
 }`}
 >
 {dfsAuto ? <><Pause className="h-3 w-3" /> : <><Play className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1);
 disabled={dfsIdx === DFS_STEPS.length - 1}
 className="p-1.5 hover:bg-slate-700 rounded"
 >
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length - 1);
 className="p-1.5 hover:bg-slate-700 rounded"
 >
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
 </div>
);
 }
}
</div>

```



```

 return <g key={n.id}>
 <circle cx={n.x} cy={n.y} r={12}
 fill={cur ? "#10b981" : st ? "#064e3b" : "#f8d7da"}
 stroke={cur ? "#fff" : st ? "#34d399" : "#f8d7da"}
 strokeWidth={2.5}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y+3} textAnchor="middle">
 {label}</text>
 {vis && (
 <div
 <rect x={n.x-18} y={n.y-28} width={36} height={56}
 className="transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor="center">
 {l:{step.low[n.id]|| "?"}}</text>
 </div>
)}
 </g>
)}
 </svg>
 <div className="flex items-center justify-between p-2">
 Шаг {dfsIdx+1}/{DFS_STEPS.length}
 <div className="flex-1 mx-2 bg-slate-950 h-10 rounded-lg">
 <div className="bg-indigo-600 h-full transition-all duration-300">
 <div>
 </div>
 </div>
 </div>
 </div>
 </div>

 {/** Code panel + stack */}
 <div className="md:col-span-2 space-y-3">
 {/** Code panel */}
 <div className="bg-slate-950 rounded-lg border border-slate-800">
 <div className="bg-slate-900 px-3 py-1.5 text-slate-400">
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono leading-relaxed">
 {CODE_LINES.map((cl, i) => {
 const hl = step.activeCodeLine.includes(cl) ? "highlight" : ""
 return <code key={i} className={hl}>{cl}</code>
 })}
 </pre>
 </div>
 </div>
 </div>
 </div>
)}
</div>

```



```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пи
 activeCodeLine: 2,
 });

 for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited }
 log: ` Выбираем вершину ${u} с мин. непосещённ
 activeCodeLine: 5,
 });
 });

```

```

 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
 }, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;

if (!currentStep) return <div className="text-white">

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-between"
 <div className="flex items-center gap-3"
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Полный
 </p>
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button
 onClick={() => { setIsPlaying(false); setCurrentStepIndex(0); }}
 className="flex items-center gap-1 bg-slate-700 text-white p-2 rounded"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}

```

```

 { /* Рёбра */ }
 { edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u);
 const vNode = nodes.find((n) => n.id === v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.u] < edge.w;

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-a)';
 else if (isOptimal) marker = 'url(#arrow-dh-n)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#f8d7da' : '#d6d8db'}
 strokeWidth={isChecking ? 5 : isOptimal ? 1 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : ''}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : '#f8d7da'}
 strokeWidth={isChecking ? 2 : 1}
 />
 <text
 x={({uNode.x + vNode.x} / 2)} y={({uNode.y + vNode.y} / 2) + 10}
 fill={isChecking ? '#f59e0b' : '#f8d7da'}
 fontSize="11" fontWeight="bold" textAnchor="center"
 >
 {edge.w}
 </text>
 </g>
);
 }) }
);
}

```



```

 </tr>
);
 }}}
 </tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60" >
 <div>
 <h4 className="text-lg font-bold text-slate-100">
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map((item) => {
 const isActive = currentStep.activeCodeLine === item.line;
 return (
 <div key={item.line} className={`py-1 px-2 ${
 isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-100'
 }`} >
 {item.line}

 {isActive && {item.code}

 </div>
);
 })}
 </div>
 </div>
 </div>
</div>
</div>
</div>
);
}

```

```

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `⚡ Кэш-хит! fibМемо(${n}) уже посчитано
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвр
 codeLine: 3,
 memoState: { ...memoMap }
 });
 return 1;
 }

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вычисляем рекурсивно: fibМемо(${n - 1})

```



```

 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
 }, [isPlaying, currentStepIndex, steps, speed]);

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
 <div className="bg-slate-900 rounded-2xl border border-slate-800" >
 /* Header & Controls */
 <div className="flex flex-col lg:flex-row lg:items-center" >
 <div>
 <div className="flex items-center gap-3 mb-2" >
 <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
 <RefreshCw className="w-6 h-6" />
 </div>
 <h3 className="text-2xl font-extrabold text-white" >
 DP
 </h3>
 <p className="text-sm text-slate-400" >
 Смотри, как таблица DP кэширует результаты
 </p>
 </div>
 </div>

 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center gap-2 bg-slate-800 p-2" >
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={!isPlaying}
 className="bg-transparent text-white font-mono"
 >

```

```

 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
 </div>
</div>

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded" >
 <h4 className="text-xs font-bold uppercase tracking-wider">DP Table View</h4>
 <div className="flex flex-wrap items-center gap-2">
 {Array.from({ length: targetN }, (_, i) => i + 1).map((num) => {
 const isCached = num in memoState || num <= 2;
 const val = num <= 2 ? 1 : memoState[num];
 const isCurrent = currentStep?.n === num;

 return (
 <div
 key={num}
 className={`flex flex-col items-center justify-center gap-2
 ${isCurrent ? 'bg-emerald-950 border-emerald-500' : ''}
 ${isCached ? 'bg-slate-900 border-emerald-500/50' : ''}
 ${!isCurrent & !isCached ? 'bg-slate-900/50 border-slate-800' : ''}
 `>
 {num}
 {isCached ? val : '∅'}
 {num <= 2 && {val}}
 </div>
);
 })}
 </div>
</div>

```

```

 {/* Tab Content */}
 <div className="min-h-[320px] flex flex-col justify-between"
 {activeTab === 'table' && {
 <div className="bg-slate-950 p-6 rounded-2xl border"
 <h5 className="font-bold text-white text-sm"
 {steps.length}
 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для N={targetN} необходимо вычислить
 вычисленное значение в таблице, мы момента.
 </p>
 <div className="p-4 bg-slate-900 rounded-xl border border-slate-300">
 Всего шагов симуляции: {steps.length}
 {steps[steps.length-1].value}
 </div>
 </div>
)}

 {activeTab === 'code' && {
 <div className="bg-slate-950 rounded-2xl border"
 <div className="bg-slate-900 px-6 py-3 border"
 {steps.length}
 {steps[steps.length-1].value}
 </div>
 <pre className="p-6 font-mono text-sm space-pre-wrap">
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2 ${
 isActive
 ? 'bg-emerald-600/20 border-l-4 border-emerald-400'
 : 'text-slate-300 hover:bg-slate-900'
 }`}
 >
 {lineNum}
 {line}
 </div>
)
 })}
 </pre>
 </div>
)}
 </div>
)

```

```
import { useState } from "react";
import { Undo } from "lucide-react";

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

const C1_EDGES = [
 { u: 0, v: 1 }, { u: 0, v: 2 },
 { u: 1, v: 3 }, { u: 2, v: 4 },
 { u: 1, v: 2 },
 { u: 3, v: 4 }
];

export function EulerSimulator() {
 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину");
 const [c1Done, setC1Done] = useState(false);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли");
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п");
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
```

```

<div className="bg-slate-950 rounded-xl p-4 border
 -hidden">
 <div className="absolute inset-0 bg-[radial-gra
 <svg className="w-full h-[260px] z-10 relative"
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!
 const v = C1_NODES.find(n => n.id === e.v)!
 const key = `${Math.min(e.u,e.v)}-${Math.ma
 const used = c1VisitedEdges.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 })}
 {C1_NODES.map(n => {
 const isLast = c1Path[c1Path.length-1] === n
 const visited = c1Path.includes(n.id);
 return <g key={n.id} className="cursor-poin
 {isLast && <circle cx={n.x} cy={n.y} r={1
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200 l
 <text x={n.x} y={n.y+4} textAnchor="middl
 bel}</text>
 </g>;
 })}
 </svg>
 <div className="absolute top-2 left-2 bg-slate-
 Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.leng
 </div>
 </div>

 <div className={`p-3 rounded-xl border text-sm ${
 c1Done && c1VisitedEdges.length === C1_EDGES.le
 ? "bg-emerald-950/40 border-emerald-500/30 te
 : c1Done

```

```
 explanation: 'Верно! Если вершины уже в одном клане
 Это нарушит структуру дерева.'
},
{
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?
options: [
 'Потому что он написан только для положительных чисел',
 'Он жадный и считает, что каждый следующий шаг должен
 закрыть вершины.',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на вершинах',
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты приедешь
 для чего нужен алгоритм Беллмана-Форда.'
},
{
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?
options: [
 'Он использует меньше памяти благодаря коммунистическим
 принципам',
 'Он умеет работать с отрицательными весами без циклов',
 'Он позволяет выполнять шаги слияния колхозов (коммуны)',
 'Он был разработан в Токио и поэтому самый быстрый',
],
 correctAnswer: 2,
 explanation: 'Абсолютно верно! В Борувке каждая коммуна
 делит вычисления на GPU или многопроцессорных кластерах',
},
{
 id: 4,
 question: 'В чем главная суть Динамического программирования?
options: [
 'Писать код очень быстро, динамично нажимая на клавиши',
 'Запоминать (кешировать) результаты уже решенных подзадач',
 'Использовать динамическую типизацию как в Python',
 'Постоянно менять требования к проекту в процессе работы'
```

```

 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
 };

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">Результаты
 <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} баллов.
 </p>

 <div className="bg-slate-950 border border-slate-900 p-12">
 <p className="text-slate-400 text-sm uppercase">Оценки
 <p className="text-5xl font-black text-indigo-400">{score}
 <p className="text-slate-300 text-sm">
 {score === quizQuestions.length
 ? ' Идеально! Ты настоящий сеньор-раскрасчик!'
 : score >= 3
 ? ' Отличный результат! Главные концепции усвоены!'
 : ' Стоит повторить главы пособия и еще раз попробовать'}
 </p>
 </div>
 </div>
);
}

```

```

let optionStyle = "bg-slate-800/80 hover:bg-
if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-ro
 } else {
 optionStyle = "bg-slate-800/40 border-s
 }
}

return {
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4"}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-
) : isAnswered && isSelected && !isCo
 <XCircle className="w-6 h-6 text-ro
) : (
 <div className={"w-6 h-6 rounded-fu
order-indigo-500 bg-indigo-500 text-white"
 {String.fromCharCode(65 + idx)}
 </div>
)}
 </div>
 <div className="flex-1 text-sm md:text-l
 {option}
 </div>
 </div>
);
}}
</div>
</div>

```



```

interface Step {
 k: number; i: number; j: number;
 matrix: number[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

 const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

 const adjList: { [key: number]: { v: number; w: numbe
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],

```

```

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), u: 0,
 log: `    Улучшение пути [${i}]→[${j}] через [${k}].`,
 activeCodeLine: 7,
 });
 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]), u: 0,
 log: `    Все пары отработаны. Алгоритм Флойда завершён.`,
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

```

```

<path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 <marker id="arrow-fw-via" markerWidth="6"
th d="M0,0 L6,3 L0,6 z" fill="#818cf8" /></
</defs>
{Object.keys(adjList).flatMap((uStr) => {
 const u = parseInt(uStr);
 return adjList[u].map((edge) => {
 const uNode = nodesSvg.find((n) => n.id
 const vNode = nodesSvg.find((n) => n.id
 const isTarget = currentStep.i === u &&
 const isVia = (currentStep.i === u && c
 let strokeColor = '#475569'; let marker
 if (isTarget) { strokeColor = '#10b981'
 else if (isVia) { strokeColor = '#818cf8
 return (
 <g key={`${u}-${edge.v}`}>
 <line x1={uNode.x} y1={uNode.y} x2=
markerEnd={marker} strokeDasharray={isTarg
 <circle cx={({uNode.x + vNode.x) / 2
isVia ? '#818cf8' : '#64748b'} strokeWidth
 <text x={({uNode.x + vNode.x) / 2} y=
8fafc'} fontSize="10" fontWeight="bold" tex
 </g>
);
 });
});
})}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a
 else if (isI) { fill = '#b45309'; stroke
 else if (isJ) { fill = '#047857'; stroke
 return (
 <g key={node.id} className="transition-
 <circle cx={node.x} cy={node.y} r={is
sJ ? 4 : 2} className="transition-colors du

```

```

)
 }
 }
 }
</div>
);
}}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Матрица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div className="flex justify-between items-center">
 <h4 className="text-lg font-bold text-rose-950">
 Матрица расстояний
 </h4>
 </div>
 <div className="overflow-x-auto">
 <table className="w-full text-center border-collapse">
 <thead><tr className="bg-rose-950/40 text-rose-950">
 <th className="p-2 border-r border-rose-950">
 Шаг
 </th>
 {nodeNames.map((n, idx) => <th key={idx} className="p-2 border-r border-rose-950 font-bold" : ''>`{n.split(' ')[0]} {idx}</th>)}
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => (
 <tr key={i} className="border-b border-rose-950">
 <td className={`p-2 bg-rose-950/40 font-bold` : ''}>
 Шаг
 </td>
 {nodeNames[i].split(' ')[0]} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== i
 const isDest = currentStep.j === j);
 let bg = 'bg-slate-900/40 text-slate-950';
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-950';
 else if (isVia) bg = 'bg-indigo-900/80 text-indigo-950';
 else if (i === j) bg = 'bg-slate-900/40 text-slate-950';
 return (<td key={j} className={`p-2 ${bg}>

```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
type Action = {
 type: 'visit' | 'process' | 'backtrack';
 node: string;
 desc: string;
 queueOrStack?: string[];
 visitedNodes?: string[];
};
```

```
function generateDFSSteps(start: string) {
```

```

 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Д

 q], visitedNodes: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, de
 tack: [...q], visitedNodes: [...Array.from(
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `04
 om(vis)] });

 return steps;
}

```

```

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("df
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);

 const steps = useMemo(() => mode === 'dfs' ? genera

 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [mode]);

 useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {

```

```

 const u = nodes.find(n => n.id === uId)
 const v = nodes.find(n => n.id === vId)
 // Check if edge is part of the current path
 const edgeTraversed = isVisited(u, v, path)

 return (
 <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 className={`stroke-${
 500 : 'stroke-emerald-500'
 } : 'stroke-slate-500'} style={strokeStyle} />
)
)
 }
)
}

{/* Nodes */}
{nodes.map(n => {
 const visited = isVisited(n.x, n.y, path)
 const current = isCurrent(n.x, n.y, path)

 let fillColor = "fill-slate-500"
 let strokeColor = "stroke-slate-500"
 let textColor = "fill-slate-500"
 let radius = 18;

 if (visited) {
 fillColor = mode === 'dark' ? 'fill-white' : 'fill-slate-500'
 strokeColor = mode === 'dark' ? 'stroke-white' : 'stroke-slate-500'
 textColor = "fill-white"
 }

 if (current && step.type !== 'edge') {
 strokeColor = "stroke-amber-500"
 textColor = "fill-amber-500"
 radius = 22;
 }

 return (
 <g key={n.id} className={`node-${
 500 : 'node-emerald-500'
 } : 'node-slate-500'}>
 <circle cx={n.x} cy={n.y} r={radius}
 className={`stroke-${
 500 : 'stroke-emerald-500'
 } : 'stroke-slate-500'} style={strokeStyle} />
 <text x={n.x} y={n.y} textAnchor="center"
 className={`text-${
 500 : 'text-emerald-500'
 } : 'text-slate-500'}>{n.id}</text>
 </g>
)
})
}

```

```

)))
 {(!step.queueOrStack || step.queue
 <div className="text-slate-500"
 })
 </div>
</div>
</div>

<div className="flex flex-col sm:flex-row gap-2"
 {/* Controls */}
 <div className="flex bg-slate-900 border border-slate-800
 <button onClick={() => {setAutoPlay(
 "p-2 text-slate-400 hover:text-white hover:bg-slate-800"
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={() => setAutoPlay(
 justify-center min-w-[80px] ${mode === 'default' ? 'play' : 'pause'}
 >
 {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
 </button>
 <button onClick={() => {setAutoPlay(
 = steps.length - 1} className="p-2 text-slate-400 hover:bg-transparent">
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={() => {setAutoPlay(
 g-slate-800 rounded-lg ml-2 border-l border-slate-800
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border border-slate-800
 ow-hidden">
 <div className="text-[10px] text-slate-400"
 {steps.length}</div>
 <div className="text-sm text-slate-400"
 {step.desc}

```



```
function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[largest], a[idx]] = [a[idx], a[largest]];
 idx = largest;
 } else break;
 }
 return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: 'Болят сердце', priority: 100 },
 { name: 'Маша (Перелом)', symptom: 'Открытый перелом ноги', priority: 80 },
 { name: 'Кот Барсик (Укус)', symptom: 'Укусил шмеля', priority: 60 },
 { name: 'Петя (Пневмония)', symptom: 'Кашель и температура', priority: 40 },
 { name: 'Оля (Диагностика)', symptom: 'Плохое самочувствие', priority: 20 },
 { name: 'Алексей (Травма)', symptom: 'Падение с высоты', priority: 10 }
];
```

```

 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const finalName = name || preset.name.split(' ')[0];
 const finalEmoji = preset.emoji;
 const finalSymptom = preset.symptom;

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 };

 const nextHeap = heapInsert(heap, newPatient).map(x => x);
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 const newPatient: Patient = {
 id: `p${uid++}`,
 name: preset.name,
 emoji: preset.emoji,
 symptom: preset.symptom,
 priority: prio,
 animState: 'in',
 };
 const nextHeap = heapInsert(heap, newPatient).map(x => x);
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === prio);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(` Поступил пациент: ${preset.name} с тяжестью ${preset.symptom}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
};

```

```

 return siftDown(a).map(x => ({ ...x, animStat:
 });

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat:
 addLog(` Успех! Пациент ${topPatient.name} по
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat:
 setLog([' База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < heap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
 x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
 stroke="#475569" strokeWidth="2"
 />

```

```

disabled={busy || heap.length >= 15}
className="flex-1 min-w-[150px] bg-gradient-to-r from-teal-600 to-teal-500
opacity-40 disabled:cursor-not-allowed text-white font-mono text-sm p-2"
ve:scale-95 transition-all cursor-pointer focus:outline-none"
>
 Привезти по Скорой (INSERT)
</button>

{/* Добавить кастомного */}
<form onSubmit={handleCustomInsert} className="flex flex-col gap-4" >
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:text-slate-400"
 />
 <div className="flex flex-col items-center min-w-[300px]" >
 Приоритет
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || !customPriority}
 className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 text-white font-mono text-sm p-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>

```

```

 const pos = nodePos(idx);
 const isRoot = idx === 0;
 return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'ani
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full r
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
 </div>
);
}}
</div>

```

```
 <div className="w-full h-3 bg-gradient-to-r f
 </div>

</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-emerald-
 {entry}
 </div>
)}}
 </div>
</div>
);
};
```

---

ФАЙЛ 42/85: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { createContext, useContext, useEffect, u
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```

const paused = useContext(DemoPauseContext);
return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза – курсор на карточке" : "наведение"}
 </div>
 </div>
);
};

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);
};

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 r: number;
 fill: string;
 stroke: string;
 label: string;
}> = ({ a, b, r, fill, stroke, label }) => (
 <g>
 <line x1={a[0]} y1={a[1]} x2={b[0]} y2={b[1]} stroke={stroke}>
 {label && (
 <text x={a[0] + b[0] / 2} y={a[1] + b[1] / 2} textAnchor="center">
 {label}
 </text>
)}
 </g>
);
};

```

```

 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
};

```



```

 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
 {step >= 2 && {
 <text
 x={({a[0] + b[0]) / 2}
 y={({a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 />
 }
 />
 />
 />
 />
 />
);
}}}
{Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
})}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1

```

```

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#f1c40f";

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "E"],
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order[step - 1] : null;
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`}>
 {GRAPH_EDGES.map(([u, v], i) => (
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
))}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k}
 fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
 stroke={wave?.includes(k) ? "#a5b4fc" : undefined} />
))}
 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={`Путь ${step}`}>
 {path.map((k, i) => (
 <Node key={k} x={GRAPH_POS[k][0]} y={GRAPH_POS[k][1]} label={k}
 fill={i === visited.length - 1 ? C.active : visited.includes(k) ? C.visited : C.inactive}
 stroke={i === visited.length - 1 ? "#a5b4fc" : undefined} />
))}
 {GRAPH_EDGES.map(([u, v], i) => (
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
))}
 </Svg>
);
};
```

```

 <text x={n.x + 30} y={n.y - 12} textAnchor="left"
 x + 1}</text>
)}
 </g>
);
})))
<text x={224} y={62} textAnchor="middle" fontSize=16>
 <text x={224} y={74} textAnchor="middle" fontSize=16>
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пусть" : ""}>
 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"}>
 <text x={130} y={54} textAnchor="middle" fontSize=16>
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize=16>
 <text x={200} y={106} textAnchor="middle" fontSize=16>
 <text x={130} y={22} textAnchor="middle" fontSize=16>
 </Svg>
);
 };
 };

```

```

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;

```

```

 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 });
 }}}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

```

```

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc

```

```

const popping = step >= 3;
return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮ tape"
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)} />
 <text x={130} y={111 - i * 26} textAnchor="middle" />
 </g>
))}
 <text x={130} y={16} textAnchor="middle" fontSize={12} />
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к концу"
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={26}
 fill={i === items.length - 1 ? C.done : C.dim} />
 <text x={53 + i * 54} y={71} textAnchor="middle" />
 </g>
))}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

```

```

 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
</Svg>
);
};

```

```

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : [];
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)})
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

```

```

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];

```

```

const a = [3, 1, 4, 1, 5, 9];
const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
const step = useLoop(4, 1200);
const l = 1, r = 3;
const show = step >= 1;
return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -
 о префиксные суммы P`}>
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim}>
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i <= r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }}>
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для
 {Array.from({ length: 8 }).map((_, i) => {

```



```

 <circle cx={196} cy={84} r={13} fill={step === 1
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
 ? "π"}</text>
 <text x={130} y={124} textAnchor="middle" font-size={12}>
 </Svг>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svг caption={step > 0 ? `π[${i}] = ${len}: префикс-суффикс` : ""}>
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.dim}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">{pi[idx]}</text>
 <rect x={18 + idx * 33} y={70} width={28} height={20}
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
))}
 <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
 <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" font-size={12}>
 π[i] — длина самого длинного «префикс = суффикс»
 </text>
 </Svг>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;

```

```
} from "./demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "./demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
```

```

}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>

```

```

export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({ top: 0, left: 0 });

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
 }, [anchor, onClose]);

 if (!anchor) return null;
 }, [anchor, onClose]);

```

```

 {entry.formal}
 </p>
 }}

 {entry.complexity && {
 <div className="text-[11px] font-mono text-
 Сложность: {entry.complexity}
 </div>
 }}

 {viz && simulatorId && {
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center
 unded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 }}
 </div>
</div>,
document.body
);
};

```

---

ФАЙЛ 49/85: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;

```

```

 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда
 return "stroke-slate-500";
 };

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {

```

```
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соеди
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mon
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.
 Старый вес: <span className="text-r
 Вычисляем: $-2 + 0 - (-2) =$ <span cl
 </div>
);
 case 5: return (
 <div>
```

```

 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 10" style="display: inline-block; width: 10px; height: 10px; vertical-align: middle; margin-right: 5px;">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 10" style="display: inline-block; width: 10px; height: 10px; vertical-align: middle; margin-right: 5px;">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 10" style="display: inline-block; width: 10px; height: 10px; vertical-align: middle; margin-right: 5px;">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 10" style="display: inline-block; width: 10px; height: 10px; vertical-align: middle; margin-right: 5px;">
 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 </defs>

```

```

 { /* Edges */ }
 { edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return {
 <g key={i}>
 <line

```



```

 {pot !== null && {
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
 }}
 </g>
)
 }}}
 </svg>
 </div>

```

```

<div className="w-full md:w-[320px] flex
 { /* Log Box */
 <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
 {getDesc()}
 </div>

```

```

 { /* Controls */
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Вар на
 <Undo strokeWidth={3} size=
 </button>
 <button onClick={() => setStep(

```

```

 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}

```

```

 null,
 false,
 -1,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 initialEdges.forEach((e) => adj[e.u].push(e.v));

 // DFS 1: post-order list
 const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Записываем в порядок`,
 u,
 false,
 -1,
);
 };

 // Run DFS1 on all unvisited
 for (let i = 0; i < 5; i++) {

```

```

 }
};

// Run DFS2 using the order (reversed from back to
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
}

recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} комп
 null,
 true,
 sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []));

```

```

<div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

```

```

<div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

```

```

 <button
 onClick={reset}
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] rela
 <svg className="w-full h-full max-w-[650px]"
 {/* Markers for directed arrows */}
 <defs>
 <marker

```

```

// If transposed, draw reversed coordinates
const x1 = step.transposed ? v.x : u.x;
const y1 = step.transposed ? v.y : u.y;
const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f4682";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 markerId={markerId}
 />
)

```

```

 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
);
}}
</svg>

```

```

{step.transposed && (
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
)}
</div>

```

```

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5

```

```


<div className="space-y-1.5 max-h-[150px]"
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map((s, idx) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded
 >
 {s.desc}
 </div>
 </div>
 </div>
)}}
 </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200"

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 text-white px-2 py-1 rounded"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 text-white px-2 py-1 rounded"
 >
 Вперед
 </button>
 </div>

```



```

}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>
 const [activeCheatTab, setActiveCheatTab] = useState<

```

```

 }, ...prev]));
 },
 // Step 4: Include D-E
 () => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
 },
 // Step 5: Inspect B-C (4)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
 },
 // Step 6: Include B-C
 () => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
 },
 // Step 7: Inspect A-D (5)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет!"',

```

```

 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'

```

```

 },
 // Step 20: Reject E-H
 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет. Вес: 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

 // --- PRIM STEPS ---
 const primSteps = [
 // Step 1: Start at E
 () => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E

```

```

// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к вершине A',
 type: 'info'
 }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
 setVertices(prev => prev.map(v => v.id === 1 ? {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 eap' } : e));
 setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (7)
 setLogs(prev => [{
 title: 'Успех: Вершина B захвачена плесенью',
 description: 'В кучу добавлено B-C(4). Ребро B-E(6)
 type: 'success'
 }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',

```

```

 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (Е) тянется к F.',
 type: 'info'
 }, ...prev]));
 },
 // Step 13: Include E-F, add F's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 id: 5,
 label: 'F',
 x: 500,
 y: 100,
 size: 100,
 color: 'black'
 } : v));
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 id: '4-5',
 source: 4,
 target: 5,
 weight: 7,
 color: 'red'
 } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (10)']);
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р.',
 type: 'success'
 }, ...prev]));
 },
 // Step 14: Pop D-G (8)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 id: '3-6',
 source: 3,
 target: 6,
 weight: 8,
 color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]));
 },
 // Step 15: Include D-G, add G's edges
 () => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 id: 6,
 label: 'G',
 x: 500,
 y: 200,
 size: 100,
 color: 'black'
 } : v));
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 id: '3-6',
 source: 3,
 target: 6,
 weight: 8,
 color: 'red'
 } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H (10)']);
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]));
 }
}

```

```

 type: 'success'
 }, ...prev]);
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]);
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]);
},
// Step 22: Pop H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 12: Куча выдает H-I (вес 12)',
 description: 'Плесень тянется к последней чистой',
 type: 'info'
 }, ...prev]);
},
// Step 23: Include H-I
() => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setHeapQueue(['F-I (13 - цикл)']));
 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф

```

```

 [0, 1, 2].includes(v.id) ? { ...v, color: 'red'
 });
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red'
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue'
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены разномасштабно из {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
},
// Step 3: Five-Year Plan 2 (Inspecting bridge between villages)
() => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мосты',
 description: 'Теперь Красный и Синий колхозы одобрены мосты A-D с весом 5. Остальные мосты B-E(6) и C-F(9) не одобрены',
 type: 'warning'
 }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
() => {
 setVertices(prev => prev.map(v => ({ ...v, color: 'blue'
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { ...e, status: 'included'
 }));

```



```

 {
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 покрашены в разные цвета. Ты можешь раскрасить
 граф, если раскраска удовлетворяет следующим
 условиям:
 - любые две смежные вершины имеют разные цвета;
 - любые две несмежные вершины имеют один и
 тот же цвет.
 Ты можешь раскрасить граф? Если да, то
 сколько существует раскрасок? Если нет,
 то почему?
 type: 'info',
 },
]});
} else if (algo === 'prim') {
 setLogs([
 {
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стар
 едь (Heap) и захватываем соседей!',
 type: 'info',
 },
]});
} else {
 setLogs([
 {
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 д
 ет имеет свой цвет. Каждая д
 ет может захватить соседей своего цвета,
 чтобы объединиться в колхоз.',
 type: 'info',
 },
]});
}
};

```

```

const getColorClass = (color: string, id: number) => {
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-white';
 case 'blue': return 'bg-blue-500 border-blue-300 text-white';
 case 'purple': return 'bg-purple-600 border-purple-300 text-white';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-white';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIndex === 0) {
 return 'bg-slate-700 border-emerald-500 text-white';
 }
 return 'bg-slate-700 border-slate-500 text-slate-300';
 }
};

```

```

 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <GitGraph className="w-3.5 h-3.5" />
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => handleReset('prim')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'prim'
 ? 'bg-emerald-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <Layers className="w-3.5 h-3.5" />
 Прима (Билет 19)
 </button>
 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-500')
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)

```

```

 { /* Legend */ }
 <div className="absolute top-4 left-4 bg-slate-500 p-4 text-white font-mono text-sm">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 </div>
 </div>
) : activeAlgo === 'prim' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-1 bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 </div>
 </div>
) : (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-500">
 </div>
 </div>
 </div>
 </div>
)
)
 </div>
 </div>
 </div>

```

```

 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
</g>
);
}}
</svg>

```

```

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full
 {vertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -
 }}
 </div>
 })}
</div>

```

```

{stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
 shadow-xl z-20">
 <p className={"font-bold text-base " +
ld-300' : 'text-rose-300'})}>

```

MST построено с помощью {activeAlgo}

```

 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCircle c
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
)))
</div>
</div>
</div>
</div>

```

```

{/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-40
 <h4 className="text-xl font-bold text-white
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button

```

```

 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 ожении:</p>
 <p className="text-xs text-slate-300 leading-5">
 ди и красим вершины в один цвет, бракуя циклы.
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-indigo-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]

def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False

2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
 if union(u, v):
 mst.append((u, v, w))`}
 </pre>
 </div>
 </div>

```

```
heap = [(0, start, -1)] # (weight, node, parent)
mst = []
```

```
while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
return mst`}
```

```
</pre>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
}}
```

```
{activeCheatTab === 'boruvka' && {
```

```
<div className="grid grid-cols-1 lg:grid-cols-
```

```
<div className="lg:col-span-1 space-y-4">
```

```
<div className="bg-slate-950 p-4 rounded-
```

```
<p className="text-slate-400 text-xs for
```

```
<Clock className="w-3.5 h-3.5 text-ro
```

```
</p>
```

```
<p className="text-2xl font-black text-r
```

```
<p className="text-slate-400 text-xs mt
```

```
p>
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-
```

```
<p className="text-slate-400 text-xs for
```

```
<Award className="w-3.5 h-3.5 text-ro
```

```
</p>
```

```
<p className="text-2xl font-black text-r
```

```
<p className="text-slate-400 text-xs mt
```

```
</div>
```

```
<div className="bg-slate-950 p-4 rounded-
```

---

ФАЙЛ 52/85: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый – зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
],
```



ФАЙЛ 53/85: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

-----

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 }, [open]);

 useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape" ?
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
 }, []);
```

```
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
)}
</>
);
};
```

---

ФАЙЛ 54/85: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cpu } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
 activeTab: 'guide' | 'simulator';
 setActiveTab: (tab: 'guide' | 'simulator') => void;
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab, compilerOpen, onToggleCompiler }) => {
 const [busy, setBusy] = useState(false);
```

```
 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
```

```

 id="tab-simulator-btn"
 onClick={() => setActiveTab('simulator')}
 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'simulator'
 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Cpu className="w-4 h-4" /> Тренажёры
 </button>
 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (на
 странице"
 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <Terminal className="w-4 h-4" /> Python
 </button>
 <a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
 title="Скачать полный PDF сборник всех стра
 >
 <FileDown className="w-4 h-4" /> Скачать PD


```

---

```
<p className="text-xs text-slate-400">Просто смотри
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-4" style={{width: 1000px; height: 1000px; position: relative; border: 1px solid #ccc; border-radius: 10px; background-color: #f0f0f0; margin: 10px auto;}}
 { /* K5 */
```

```
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800" style={{width: 100%; height: 100%; position: relative; background-color: #000000; color: #ffffff; font-family: sans-serif; font-size: 14px; margin: 10px auto;}}>
```

```
 <div className="absolute top-2 left-2 bg-slate-900 p-2" style={{width: 100%; height: 100%; position: relative; background-color: #000000; color: #ffffff; font-family: sans-serif; font-size: 14px; margin: 10px auto;}}>
 K5 – НЕПЛАНАРЕН</div>
```

```
 <svg className="w-full h-full absolute inset-0" style={{width: 100%; height: 100%; position: relative; background-color: #000000; color: #ffffff; font-family: sans-serif; font-size: 14px; margin: 10px auto;}}>
 {(() => {
```

```
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
```

```
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
```

```
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
```

```
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
```

```
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
```

```
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
```

```
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 }
 }
 }
</div>
```

```

 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.f[id]; }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 const labels = ["\u04141","\u04142","\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" stroke="#4f46e5" stroke-width={2}/>
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id-1]}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>

```

```

import { getPageSync } from "../data/vizSync";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Перейти к странице пособия (посмотреть визуализацию) */
 onOpenGuide: () => void;
 /** Скрыть панель. */
 onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
 }
 return runtimePromise;
}
```

```

/* Шпаргалка Python: команды сортировки, циклы, структуры
/* «Вставить» кладёт сниппет в редактор под курсор.
/* -----
```

```
interface Snippet {
 label: string;
 tip: string;
 code: string;
}
```

```
const SNIPPETS: { group: string; items: Snippet[] }[] =
 [
 {
 group: "Сортировка",
 items: [
 {
 label: "arr.sort() / sorted()",
 tip: "In-place сортировка по возрастанию и новая
 code: 'arr = [5, 2, 8, 1]\narr.sort()
) # копия по убыванию\n',
 },
 {
 label: "sorted(key=...)",
 tip: "Сортировка по ключу: рёбра по весу, пары
 code: 'edges = [(3, "A", "B"), (1, "B", "C"), (3, "C", "A")]\nedges\n',
 },
],
 },
 {
 group: "Циклы",
 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы
 code: "n, m = 3, 4\nfor i in range(n):
 for j in range(m):
 ...",
 },
 {
 label: "while ...",
 },
],
 },
]
```

```

 code: 'visited = set()\nvisited.add("A")\nprint
 },
],
},
{
 group: "Вывод",
 items: [
 {
 label: "f-строка",
 tip: "Подставить значения переменных шага в строку",
 code: 'i, j, k = 1, 2, 0\nprint(f"dist[{i}][{j}] = {k}")\n',
 },
],
},
];

```

/\*\* Переживает скрытие панели: код пользователя не теряется

```

let savedEditor: { code: string; template: string; chapterId: string };

```

```

export function PythonCompiler({ chapterId, chapterTitle }: ChapterProps) {
 const sync = useMemo(() => getPageSync(chapterId), [chapterId]);
 const template = useMemo(() => buildSyncTemplate(chapterId, chapterTitle));

 const [code, setCode] = useState(() => {
 if (!savedEditor) return template;
 if (savedEditor.code === savedEditor.template || savedEditor.chapterId !== chapterId)
 return savedEditor.code;
 });

 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");
 const [copied, setCopied] = useState(false);
 const [stripTab, setStripTab] = useState<"vars" | "highlight">("vars");
 const [outputOpen, setOutputOpen] = useState(true);
 const editorRef = useRef<HTMLTextAreaElement>(null);

```



```

 if (running) return;

 setRunning(true);
 setOutputOpen(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." :

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = stdin.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов – код выполняется

 runtime.setStdout({ batched: (message) => stdout.push(message)});
 runtime.setStderr({ batched: (message) => stderr.push(message)});
 runtime.setStdin({ stdin: () => {inputLines.length > 0 ? inputLines.shift() : null}});
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не вывела");
} catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}Ошибка: ${error.message}`);
 setRuntimeMessage("Не удалось выполнить – проверь код");
} finally {
 setRunning(false);
}
}, [code, stdin, running, runtimeReady]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {

```

```

 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-slate-400" /> : <Copy className="w-4 h-4 text-slate-400" />}
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Вернуть коммент" >
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скрыть панель" >
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Запустить код" >
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="ml-1 inline-flex items-center justify-center gap-2 rounded-lg text-slate-400 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : <Play className="w-3.5 h-3.5" />}
 </button>
 </Tooltip>
 </div>
)
}

```

```

 className={`cursor-help inline-flex items-center justify-center
 hasViz ? "text-emerald-400" : "text-slate-400"
 }`}
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визита"}

 </Tooltip>
</div>
</div>

{stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-w-300">
 <Tooltip side="bottom" content="Открыть эскиз">
 <button
 type="button"
 onClick={onOpenGuide}
 className="inline-flex items-center gap-2 px-2 py-1.5
 text-slate-300 hover:text-white hover:bg-slate-500"
 >
 {chapterTitle}
 </button>
 </Tooltip>
 </div>
 {desyncedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница
 была заменена)."
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5 px-2 py-1.5
 text-amber-400 hover:bg-amber-500/20"
 >
 <Unlink className="w-3.5 h-3.5 shrink-0" />
 </button>
 </Tooltip>
)}
 </div>
) : null}
```

```


 </Tooltip>
 })
 </div>
})
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow
 {SNIPPETS.map(({gItem}) => {
 <div key={gItem.group}>
 <div className="text-[9px] font-bold up
 <div className="flex flex-wrap gap-1.5"
 {gItem.items.map(({s}) => {
 <Tooltip key={s.label} side="bottom
 <button
 type="button"
 onClick={() => insertSnippet(s.
 className="inline-flex items-ce
 -[10.5px] text-indigo-300 hover:text-white
 title=""
 >
 <Plus className="w-3 h-3 text-s
 {s.label}
 </button>
 </Tooltip>
 }}}
 </div>
 </div>
 </div>
 })}
 <p className="text-[10px] text-slate-600">H
 </div>
})
</div>

```

```

{/* — Редактор + ввод —————
<div className="flex-1 min-h-0 flex flex-col">
 <textarea
 ref={editorRef}

```

```

 <Terminal className="w-3.5 h-3.5 text-emerald-500 font-mono">
 {outputOpen && {output}}
 </Terminal>

 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-500 font-mono">
 {outputOpen && {output}}
 </CheckCircle2>

 </button>
 {outputOpen && (
 <div className="flex-1 min-h-[70px] overflow-hidden leading-5 text-slate-300">
 {output}
 </div>
 <div className="px-3 py-1.5 border-t border-slate-200 border-b border-slate-200">
 {output}
 </div>
)}
 </div>
</aside>
);
}
```

---

ФАЙЛ 57/85: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';
```

```
interface Customer {
 id: string;
 name: string;
 emoji: string;
 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}
```

```

 if (isServing) return;
 if (queue.length >= 6) {
 addLog('△ Хвост очереди уже на улице, повару тяже
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PRESETS[counter % PRESETS.length];
 counter++;

 const newGuest: Customer = {
 id: Date.now().toString(),
 ...preset,
 animState: 'in',
 };

 setQueue(prev => [...prev, newGuest]);
 addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

 setTimeout(() => {
 setQueue(prev => prev.map(c => c.id === newGuest.
 }, 450);
 }, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('△ DEQUEUE: В очереди никого нет! Повар ск
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

```

```

<div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-indigo-950/40 border border-in
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-indigo-400 tex
 <p className="text-xs text-slate-300 mt-1 lea
 В очереди за супом всё честно. Кто первый в
 Новые гости встают строго в конец оч
 Здесь нет обхода и блата – только строгий п
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap
 <button
 onClick={enqueue}
 disabled={isServing}
 className="flex-1 min-w-[150px] bg-gradient-t
 d:opacity-40 disabled:cursor-not-allowed te
 ive:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-t
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}

```

```

 { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
 <div className="flex flex-col items-center" >

 </div>

 { /* СПИСОК ОЧЕРЕДИ */ }
 <div className="flex-1 flex items-end gap-4" >
 {queue.length === 0 ? (
 <div className="flex-1 flex flex-col items-center" >
 </
 <p className="text-xs font-bold font" >
 <p className="text-[10px]" >Все сыты
 </div>
) : (
 queue.map((customer, idx) => {
 const isHead = idx === 0;
 const isTail = idx === queue.length - 1;
 return (
 <div
 key={customer.id}
 className={`flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `
 >
 { /* Облачко мыслей */ }
 <div className={`absolute -top-16 left-1/2 -transform
 8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border' : ''}
 `} >
 <div className="font-bold text" >
 {customer.bubbleText}
 <div className="absolute -bottom-16 left-1/2
 it rotate-45"></div>
 </div>
 </div>
)
 })
)
 </div>

```



```

 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse">
 СЫТЫ
 </div>

 <div className="flex-1 flex flex-col justify-between p-4" >
 {servedHistory.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center" >

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-1.5 w-full" >
 {servedHistory.map(({item, idx}) => (
 <div
 key={idx}
 className="w-full py-1.5 px-3 rounded-
0 flex items-center gap-2 text-[11px] text-slate-
 >
 {item}
 </div>
)}}
 </div>
)}
 </div>

 {/* Пол */}
 <div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800" >
 </div>
 </div>

 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800 p-4" >
 <div className="text-[10px] font-mono text-slate-400" >

```

```

const [words, setWords] = useState<string[]>(['CAR',
const [newWord, setNewWord] = useState('');
const [nodes, setNodes] = useState<{ [key: number]: T
const [simulationStep, setSimulationStep] = useState<
const [bfsQueue, setBfsQueue] = useState<number[]>([
const [currentNode, setCurrentNode] = useState<number
const [speechText, setSpeechText] = useState<string>('
 'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
 буль!')
});
const [isTalking, setIsTalking] = useState<boolean>(f
const [isBlinking, setIsBlinking] = useState<boolean>

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);
 }, 4000);
 return () => clearInterval(blinkInterval);
}, []);

// Эффект разговора при смене текста
useEffect(() => {
 setIsTalking(true);
 const timer = setTimeout(() => setIsTalking(false),
 return () => clearTimeout(timer);
}, [speechText]);

// Построение начального бора (Trie) без fail-ссылок
const buildInitialTrie = (wordList: string[]) => {
 const newNodes: { [key: number]: TrieNode } = {
 0: { id: 0, char: 'ROOT', parent: 0, children: {}
 };
 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;

```

```

 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * 2 : rightX;
 if (leftX === -1) currentX++;
 }
};

layoutDFS(0, 0);

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
 `Отлично! Я построил базовое префиксное дерево (БПД) для слова "Базовое",
 чтобы расставить fail-ссылки!`
);
};

useEffect(() => {
 buildInitialTrie(words);
}, []);

const handleAddWord = (e: React.FormEvent) => {
 e.preventDefault();
 const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
 if (!cleanWord) return;
 if (words.includes(cleanWord)) {
 setSpeechText(`Слово «${cleanWord}» уже есть в слове "Базовое".`);
 return;
 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
};

```

```

 'Ура! Очередь пуста! Мы успешно построили полный
 ь полюбоваться таблицей переходов!'
);
 return;
}

const r = bfsQueue[0];
const newQueue = bfsQueue.slice(1);
const updatedNodes = { ...nodes };
const rNode = updatedNodes[r];
const childrenIds = Object.values(rNode.children);

let logs = `Рассматриваем вершину #${r} (${rNode.chi

for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
 v = updatedNodes[v].fail;
 }
 const failTarget = updatedNodes[v].children[char];
 updatedNodes[u].fail = failTarget;

 // Рассчитываем терминальную ссылку (term_link)
 if (updatedNodes[failTarget].is_terminal) {
 updatedNodes[u].term_link = failTarget;
 } else {
 updatedNodes[u].term_link = updatedNodes[failTa
 }

 logs += `Для #${u} ('${char}') fail = #${failTarg
}

if (childrenIds.length === 0) {
 logs += 'У этой вершины нет детей, просто переход
}

setNodes(updatedNodes);

```

```

 { /* SVG Лосося */ }
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_10px]_#000' : ''
 }`
 >
 { /* Тело лосося */ }
 <ellipse cx="100" cy="100" rx="70" ry="45" fill="#e67e22" />
 <path d="M 40 85 Q 100 55 160 85 Q 100 115 40 85" fill="#e67e22" />

 { /* Хвост */ }
 <path
 d="M 35 100 L 5 70 L 15 100 L 5 130 Z"
 fill="#be123c"
 className="origin-[35px_100px] animate-pulse"
 />

 { /* Плавники */ }
 <path d="M 90 55 L 70 30 L 110 55 Z" fill="#000000" />
 <path d="M 90 145 L 70 170 L 110 145 Z" fill="#000000" />
 <path
 d="M 120 115 L 100 135 L 130 130 Z"
 fill="#f87171"
 className="origin-[120px_115px] animate-pulse"
 />

 { /* Глаз (с морганием) */ }
 <circle cx="145" cy="85" r="12" fill="#fff" />
 {isBlinking ? (
 <line x1="133" y1="85" x2="157" y2="85" />
) : (
 <circle cx="148" cy="85" r="6" fill="#000" />
 <circle cx="151" cy="82" r="2" fill="#fff" />
)}
 </svg>
)}

```

```

<div className="text-xs text-slate-400 flex" style={{width: 100px, height: 100px; border: 1px solid #ccc; border-radius: 5px; display: flex; align-items: center; justify-content: center; background-color: #f0f0f0; margin: 10px auto; text-align: center; font-family: sans-serif; font-size: 0.8em; color: #333; position: relative; overflow: hidden; box-shadow: 0 2px 4px #ccc; transition: all 0.2s ease;}}
 <Info className="w-3.5 h-3.5 text-blue-400" style={{width: 100px, height: 100px; border: 1px solid #ccc; border-radius: 5px; display: flex; align-items: center; justify-content: center; background-color: #e0f0ff; margin: 10px auto; text-align: center; font-family: sans-serif; font-size: 0.8em; color: #007bff; position: relative; overflow: hidden; box-shadow: 0 2px 4px #add8e6; transition: all 0.2s ease;}}
 Статус: {simulationStep === 0 ? 'Бор готов к запуску!' : 'Бор работает!'}
 </div>

<div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-700 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-transform active:scale-95"
 >
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stepBFS}
 className="bg-blue-600 hover:bg-blue-700 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-transform active:scale-95"
 >
 Шаг BFS

 {currentStep}

 </button>
)}
 {simulationStep === 2 && (
 <button
 onClick={() => buildInitialTrie(words)}
 className="bg-indigo-600 hover:bg-indigo-700 text-white px-4 py-2 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
 >
 <RotateCcw className="w-4 h-4" /> Перезагрузить
 </button>
)}
</div>
</div>
</div>

```

```

 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#ccc'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={({parent.x + node.x} / 2 - 10)}
 y={({parent.y + node.y} / 2)}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}
 </text>
 </g>
);
 }}}

 {/* Draw Fail Links */}
 {simulationStep > 0 && Object.values(nodes).some(
 node => node.fail !== 0
) ? (
 // Wait, all nodes get fail link. Let's draw them all! But maybe wait, we can just draw the first one
 // To avoid too many lines, let's draw the first one
 const target = nodes[nodes[0].fail];
 if (!target || !target.x || !target.y || !target.char) return;

 // Simple bezier curve for fail link
 const isCurrent = currentNode === nodes[0];
 const path =

```

```

 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && {
 <text
 x={node.x + 12}
 y={node.y - 12}
 fill="#94a3b8"
 fontSize="9"
 >
 {node.id}
 </text>
 }}
 </g>
);
 }}}
</svg>
);
}}())
</div>

{/* Управление словарем */}
<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl flex flex-col">
 <div>
 <h5 className="text-white font-bold mb-3 flex items-center">

```



```

 </button>
 </form>
 </div>

 { /* Таблица Вершин и суффиксных ссылок */ }
 <div className="lg:col-span-2 bg-slate-900/50 p-4">
 <h5 className="text-white font-bold mb-3 flex items-center">

 Таблица переходов и fail-ссылок

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-slate-700">
 <th className="py-2 px-3">ID Вершины</th>
 <th className="py-2 px-3">Символ</th>
 <th className="py-2 px-3">Родитель</th>
 <th className="py-2 px-3">fail-ссылка</th>
 <th className="py-2 px-3">term_link</th>
 <th className="py-2 px-3">Дети</th>
 <th className="py-2 px-3">Слова (dict)</th>
 </tr>
 </thead>
 <tbody className="divide-y divide-slate-800">
 {Object.values(nodes).map((node) => {
 const isCurrent = currentNode === node
 const isInQueue = bfsQueue.includes(node.id)

 return (
 <tr>
 <td>{node.id}</td>
 <td>{node.symbol}</td>
 <td>{node.parent}</td>
 <td>{node.fail}</td>
 <td>{node.term_link}</td>
 <td>{node.children}</td>
 <td>{node.words}</td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
 </div>
 </div>
</div>

```

```

<td className="py-2.5 px-3">
 {node.id === 0 || node.term_link

) : (
 <span className="bg-rose-950 text-white transition-colors cursor-help" title={`Ск
 #{node.term_link}

)}
 </td>
<td className="py-2.5 px-3 text-slate-500">
 {Object.keys(node.children).length === 0 ? 'Ø' : Object.entries(node.children)
 .map(([c, id]) => `${c}→#${id}`)
 .join(', ')}
 </td>
<td className="py-2.5 px-3">
 {node.words.length === 0 ? (

) : (

 {node.words.join(', ')}

)}
 </td>
</tr>
</div>
</tbody>
</table>
</div>
</div>
</div>
</div>
);
};

```

```
// 1. Build steps (recursive top-down)
function genBuildSteps(arr: number[]): Step[] {
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};

 const CODE = [
 "build(v, l, r) {", // 1
 " if (l === r) {", // 2
 " tree[v] = A[l]; return", // 3
 " }", // 4
 " mid = (l + r) >> 1", // 5
 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
 void CODE;

 function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
 highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[2 * v] + tree[2 * v + 1]` });
 }
}
```

```

 highlightNodes: [1], arrayHighlight: [qL, qR], result
 return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree

```
function genQueryBottomUpSteps(arr: number[], qL: number
```

```
 const n = arr.length;
```

```
 // Build iterative tree: leaves at [n..2n-1]
```

```
 const tree: Record<number, number> = {};
```

```
 for (let i = 0; i < n; i++) tree[n + i] = arr[i];
```

```
 for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i
```

```
 const steps: Step[] = [];
```

```
 let id = 0;
```

```
 let l = qL + n;
```

```
 let r = qR + n + 1; // half-open [l, r)
```

```
 let res = 0;
```

```
 steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL
```

```
 `, codeLine: 3, treeState: tree, highlightNodes: [l
```

```
 while (l < r) {
```

```
 const highlights: number[] = [];
```

```
 if (l & 1) {
```

```
 res += tree[l] ?? 0;
```

```
 steps.push({ id: id++, desc: `l=${l} нечётный → l
```

```
 0}. Сумма = ${res}. l++.`, codeLine: 5, treeSta
```

```
 highlights.push(l);
```

```
 l++;
```

```
 }
```

```
 if (r & 1) {
```

```
 r--;
```

```
 res += tree[r] ?? 0;
```

```
 steps.push({ id: id++, desc: `r=${r + 1} нечётный
```

```
 , treeState: tree, highlightNodes: [r], arrayHi
```

```
 highlights.push(r);
```

```
 }
```

```
 l >>= 1;
```

```

 " // -----",
 " mid = (l + r) >> 1;",
 " // спускаемся в обоих детей",
 " return query(left) + query(right);",
 "}"
];
const QUERY_BU_CODE = [
 "queryBU(ql, qr) {",
 " res = 0;",
 " l = ql + n; r = qr + n + 1;",
 " // -----",
 " while (l < r) {",
 " if (l & 1) res += tree[l++];",
 " // -----",
 " if (r & 1) res += tree[--r];",
 " l >>= 1; r >>= 1;",
 " }",
 " return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500',
 border: 'border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-600'
 }
};

```

```

 <div className="flex flex-col items-center mt-2" >
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full" >
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
</div>
);
}, [step, clr]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 }/* Header */>
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-b-2 border-b-amber-500` >
 <h4 className="font-bold text-sm text-white flex-1" >

 </div>

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b-2 border-b-amber-500" >
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.start
 return (
 <div key={i} className={`flex flex-col items-center justify-between
 go' ? 'bg-indigo-950 border-indigo-500' : 'bg-slate-900 border-amber-500'
 -translate-y-0.5` : 'bg-slate-900 border-amber-500'` >

 </div>
);
 })}
 </div>

 {/* Tree */}

```

```

 <button onClick={() => { setPlaying(false); s
 0 hover:text-white hover:bg-slate-800 round
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-5
 {playing ? <Pause className="w-3 h-3" /> :
 </button>
 <button onClick={() => { setPlaying(false); s
 .5 text-slate-400 hover:text-white hover:bg
 utton>
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 {/* Progress bar */}
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
</div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12
 const [customInput, setCustomInput] = useState('5, 8,
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'

 // Clamp query range when array changes
 useEffect(() => {

```





```

 { /* ---- TAB CONTENT ---- */
 {view === 'build' && {
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
 }}

 {view === 'queries' && {
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑ "
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`
 icon="↑ "
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
 }}

 {view === 'theory' && {
 <div className="space-y-5">

```

```

 d-400 font-bold">✓ легко</div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl b
 <div className="absolute -top-3 left-4 bg
rder border-amber-500 shadow-md">
 1 Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3
 Стартуем с двух указателей (l и r) на л
ём его левого соседа. Поднимаемся пока l {"
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between bo
span className="text-emerald-400 font-mono":
 <div className="flex justify-between bo
><span className="text-emerald-400 font-mono
 <div className="flex justify-between"><
00 font-bold">x сложно</div>
 </div>
 </div>
 </div>

 {/* Verdict */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="font-bold text-blue-400 mb-2
 <p className="text-sm text-slate-300 leadin
 <strong className="text-white">Итеративны
кальности. Но если нужны <strong className=
ame="text-white">рекурсивный (сверху-вниз)<
 </p>
 </div>
</div>
 })
</div>
);
}

```

```

</h3>
<div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute" />
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border border-indigo-500 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -translate-y-1/2 text-indigo-500 hover:text-indigo-600"
 aria-label="Очистить поиск"
 />
)}
</div>
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4">
 {groups.length === 0 && <p className="text-sm text-slate-500">Нет тем</p>}

 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold uppercase">{category}</div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 }}
 />
)
 })}
 </div>
 </div>
)}
</div>

```

```
 </div>
 </aside>
);
 };
};
```

---

ФАЙЛ 61/85: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, ArrowLeft } from 'lucide-react';

interface Plate {
 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;
 name: string;
 icon: string;
}

interface Hypothesis {
 id: string;
 text: string;
 probability: number;
}

export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' | 'hypotheses'>('stack');

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
])
```

```

 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate}
 `);

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate}
 `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🥘' },
 { name: 'Курьер доставки', icon: '📦' },
 { name: 'Усталый админ', icon: '🤖' },
 { name: 'Турист с рюкзаком', icon: '🎒' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
};

```

```

 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 `);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-800 pb-6 mb-6">
 <div className="border-b border-slate-800 pb-6 mb-6">
 <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-slate-400 to-slate-600">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритетной очереди
 </p>
 </div>
 </div>

```

```


 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 text-emerald-500'
 : 'bg-slate-800/60 border-slate-700/60 text-slate-400'
 }`>
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'text-emerald-500' : 'text-slate-400'} fill-current` />
 <div className="text-left">
 <div className="font-bold text-sm text-white">Горю
 <div className="text-xs opacity-80">Приоритетная очередь
 </div>
 </div>

 Beam Search

 </button>
 </div>

 { /* STACK TAB */ }
 {activeTab === 'stack' && (
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-xl text-slate-400 justify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-white">Симуляция стопки тарелок
 Симуляция стопки тарелок

 </h3>
 <p className="text-slate-400 text-xs mt-1">Новые тарелки ставятся наверх. Мыть мож

```





```

 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
</div>
</div>

{dequeueMessage && (
 <div className="bg-indigo-950/60 border bor
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>
)}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очер
 <p className="text-xs mt-1">Добавьте ро
 </div>
) : (
 <div className="w-full flex items-center
 <div className="flex flex-col items-cen
300">


```

```

 <div className="flex flex-col items-center
-600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
 </div>
 })
 <div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
 </div>
</div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 <div className="flex items-center gap-3 w-f
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex ite
 py-2.5 rounded-xl text-sm font-bold shadow-
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button

```

```

 <div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center
 bg-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
 </div>
 </form>

 {heapMessage && {
 <div className="bg-emerald-950/60 border border-emerald-500/60
 gap-3 animate-fade-in">
 <span className="inline-block w-2 h-2 rounded-full" style={{
 background-color: heapMessage.color
 }} />
 {heapMessage}
 </div>
 }}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2xl">
 <div>
 {sortedHeap.length === 0 ? (
 <div className="text-center py-12 text-slate-400">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Куча пуста</p>
 <p className="text-xs mt-1">Добавьте гирю</p>
 </div>
) : (
 <div className="w-full max-w-2xl space-y-4">
 <div className="flex items-center justify-between">
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>
 {sortedHeap.map((item, index) => {

```

```
 </div>
 <span className={`font-mono font-size-400
 isTop ? 'text-emerald-400 text-emerald-500' : 'text-slate-500'}>
 {percent}%

 </div>
</div>
);
 }
</div>
)
 <div className="mt-6 text-xs text-slate-500">
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
 </div>
</div>
</div>
)
</div>
);
};
```

---

ФАЙЛ 62/85: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```

```
interface Props {
 /** Открыть тренажёр в модальном окне. */
 onOpen: (vizId: string) => void;
 /** Перейти к теме, к которой привязан тренажёр. */
 onGoChapter: (chapterId: string) => void;
```

```

<div className="relative mb-6 max-w-md">
 <Search className="w-4 h-4 text-slate-500 absolute"
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск тренажёра..."
 className="w-full bg-slate-900 border border-
 focus:outline-none focus:border-indigo-500"
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -transl
 aria-label="Очистить поиск"
 >
 <X className="w-4 h-4" />
 </button>
)}
 </div>

```

```

{items.length === 0 && <p className="text-slate-5

```

```

<div className="grid gap-3 sm:grid-cols-2 xl:grid
 {items.map(({ id, entry, chapterTitle }) => (
 <div
 key={id}
 className="flex flex-col bg-slate-900 borde
 >
 <h3 className="font-bold text-white text-sm
 {entry.hint && <p className="text-[12px] te
 {!entry.hint && <div className="flex-1" />}
 </h3>
 <div className="flex items-center gap-2 mt-
 <button
 onClick={() => onOpen(id)}
 className="flex items-center gap-1.5 te
 transition-colors"
 >

```

```

export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
 {
 name: "Один уровень",

```

```

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
 }

 /** Какой поворот сделал бы настоящий splay из текущего
 export function canonicalNext(root: TNode, key: number) {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя" };
 }

 /* ----- раскладка -----

 interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
 }

```

```

const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target))
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target));

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.key === key }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

```



```

<svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
>
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 })}

 {nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >

```

```

 ? "bg-amber-600/20 border-amber-500 text-slate-500"
 : "bg-slate-800 border-slate-700 text-slate-500"
 }`}
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : null}
 {showAnswer ? "Скрыть разбор" : "Сдаться / посмотреть"}
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

</div>

{showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-2">
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

{solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-500"
 >
 <div className="flex items-center gap-2 font-mono">
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : null}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`}
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>

```

## Универсальное пособие • полный исходный код

```

* * по умолчанию анимация НЕ крутится сама: пользователь
* идёт в своём темпе. Автопрокрутка включается кнопкой
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел». */
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стрелку. */
 moved?: NodeId[];
 ms: number;
}
```

```
interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах. */
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}
```

```
const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,
```

```

 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C: [160, 112],
 edges: [["g", "p"], ["g", "D"], ["p", "x"], ["p", "C"],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g: [255, 40],
 edges: [["p", "x"], ["p", "g"], ["x", "A"], ["x", "B"],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",

```

```
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p: [148, 112] },
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"], ["p", "D"], ["p", "C"]],
} as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p — дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым — нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 — 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 }
]
}
```

```

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(
 const ghosts = frame.kind === "result" ? (moved ?? [])

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fil
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
)
)
)}
)

```

```

 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke=
 <circle r={r} fill={c.fill} stroke={c.stroke}
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font=
 {id}
 </text>
)}
 </g>
);
}})
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

 const active = CASES[caseIdx];
 const total = active.frames.length;
 const idx = Math.min(frame, total - 1);
 const current = active.frames[idx];
 const prevPos = active.frames[Math.max(idx - 1, 0)].p
 const duration = useMemo(() => Math.round(current.ms

```

```

 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }` }
 >
 {current.kind === "aim" ? "прицел" : current.title}

 Шаг {idx + 1} из {total}. {current.title}

</div>
<p className="text-[13px] text-slate-400 leading-tight">
</div>

<div className="bg-slate-900 rounded-xl border border-slate-800">
 <Tree frame={current} prev={prevPos} keys={activeKeys}>

 {/* обход слева направо – одинаковый на всех кадрах */}
 <div className="mt-2 pt-2 border-t border-slate-800">
 <div className="flex items-center gap-1.5 flex-wrap">
 Обход
 {active.inorder.map((t, i) => {
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-mono text-sm
 /\d/.test(t) ? "bg-emerald-500/15 text-white" : "bg-slate-800/50 text-slate-400"}
 >
 {t}

))}
 Отображение
 </div>
 <p className="text-[11px] text-slate-500 mt-1">
 </div>
 </div>

```



```

 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-3
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500
 : "bg-slate-800 border-slate-700 text-slate-200
 }`}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.1" : "0.5"}
 </button>
 <button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-3
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
 </button>

 <div className="flex items-center gap-3 ml-auto"
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : (
 <span key={id} className="flex items-center justify-center"
 <span className="w-3 h-3 rounded-full" style={{
 background: active[id],
 }} />

)
)}
 </div>
</div>

```

```
};
```

```
// Right Rotation (Zig / Right Rotate)
```

```
const rightRotate = (x: SplayNode): SplayNode => {
 const y = x.left;
 if (!y) return x;
 x.left = y.right;
 y.right = x;
 return y;
};
```

```
// Left Rotation (Zag / Left Rotate)
```

```
const leftRotate = (x: SplayNode): SplayNode => {
 const y = x.right;
 if (!y) return x;
 x.right = y.left;
 y.left = x;
 return y;
};
```

```
// Clone tree helper
```

```
const cloneTree = (node: SplayNode | null): SplayNode |
 if (!node) return null;
 return {
 key: node.key,
 left: cloneTree(node.left),
 right: cloneTree(node.right),
 x: node.x,
 y: node.y,
 };
};
```

```
export const SplayTreeViz: React.FC = () => {
```

```
 // Let's create an elegant initial unbalanced or semi
```

```
 // Values: 10, 20, 30, 40, 50, 60
```

```
 const createInitialTree = (): SplayNode => {
```

```
 let r: SplayNode | null = null;
```

```
 const initialKeys = [40, 20, 50, 10, 30, 60];
```

```

 current = current.right;
 }
}

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
 steps.push(`Поиск ${key}: элемент отсутствует в д
 steps.push(
 `Поиск споткнулся на узле [${lastNode.key}]. И
);
} else {
 steps.push(
 `Поиск ${key}: элемент найден! Запускаем подъем
);
}

// Now let's implement standard Splay logic recursively
const splayAction = (
 node: SplayNode | null,
 k: number,
): SplayNode | null => {
 if (!node || node.key === k) return node;

 if (k < node.key) {
 if (!node.left) return node;

 if (k < node.left.key) {
 // Zig-Zig (Left-Left)
 node.left.left = splayAction(node.left.left,
 node = rightRotate(node);
 steps.push(
 `Вращение Zig-Zig (Правый поворот родителя
);
 } else if (k > node.left.key) {
 // Zig-Zag (Left-Right)
 node.left.right = splayAction(node.left.right

```

```

 const finalRoot = splayAction(cloneTree(root), target);
 return { newRoot: finalRoot, steps };
 };

```

```

const handleSplay = (key: number) => {
 setHighlightedNode(key);
 const { newRoot, steps } = splayStepByStep(tree, key);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog(steps);
};

```

```

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;

 // Standard BST insert, then splay!
 let newTree = cloneTree(tree);
 newTree = insertBST(newTree, val);
 const { newRoot, steps } = splayStepByStep(newTree, val);
 if (newRoot) {
 setTree(newRoot);
 }
 setLog([`Вставили вершину [${val}].`, ...steps]);
 setInputValue("");
 setHighlightedNode(val);
};

```

```

const handleFind = (e: React.FormEvent) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (isNaN(val)) return;
 handleSplay(val);
 setInputValue("");
};

```

```

 x2={node.left.x}
 y2={node.left.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.left));
 }
 if (node.right && node.right.x && node.right.y) {
 lines.push(
 <line
 key={`l-${node.key}-${node.right.key}`}
 x1={node.x}
 y1={node.y}
 x2={node.right.x}
 y2={node.right.y}
 stroke="#475569"
 strokeWidth="2"
 />,
);
 lines = lines.concat(renderLines(node.right));
 }
 return lines;
};

```

```

const renderNodes = (node: SplayNode | null): React.ReactNode[] => {
 if (!node) return [];
 let circles: React.ReactNode[] = [];
 const isHighlighted = highlightedNode === node.key;

 circles.push(
 <g
 key={`n-${node.key}`}
 className="cursor-pointer group"
 onClick={() => handleSplay(node.key)}
 data-title={`Кликните, чтобы выполнить Splay(${node.key})`}
 >
 <circle

```

```

 </p>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12" >
 { /* Playfield */ }
 <div className="lg:col-span-7 bg-[#0f172a] p-4" >
 <svg
 className="w-full h-full max-w-[600px] max-h-[320px]"
 viewBox="0 0 600 320"
 >
 {renderLines(drawTree)}
 {renderNodes(drawTree)}
 </svg>

 { /* Quick interactive controls */ }
 <div className="absolute bottom-4 left-4 right-4" >
 <form onSubmit={handleFind} className="flex" >
 <input
 type="number"
 placeholder="Поиск..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 className="w-24 bg-slate-950 text-white"
 />
 <button
 type="submit"
 onClick={handleFind}
 className="bg-indigo-600 hover:bg-indigo-500"
 >
 Поиск / Splay
 </button>
 <button
 type="button"
 onClick={(e) => {
 e.preventDefault();
 const val = parseInt(inputValue);
 if (!isNaN(val)) {

```

```

 : "text-slate-400 bg-slate-900/50"
 }` }
 >
 {step}
 </div>
)}}
</div>

<div className="border-t border-slate-800 pt-10">
 <p>

 у самого корня. Делаем 1 вращение.

 </p>
 <p>

 оба левые или оба правые. Вращаем деда, по

 </p>
 <p>

 колено. Вращаем узел в разные стороны.

 </p>
</div>
</div>
</div>

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
 <h4 className="text-indigo-400 font-bold text-slate-400">
 <HelpCircle className="w-4 h-4 text-indigo-400"/>
 Разбор экзаменационных вопросов (Без нитья):
 </h4>

 <div className="grid grid-cols-1 md:grid-cols-3">
 <div className="bg-slate-900/60 p-4 rounded-l">
 <div>
 <p className="font-bold text-slate-300 mb-2">
 1. Отсутствующий элемент
 </p>

```

```

 Кэшировать свингующие пары на концах — фа
 </div>
 </div>

 <div className="bg-slate-900/60 p-4 rounded-l
 <div>
 <p className="font-bold text-slate-300 mb
 3. Амортизация $O(\log n)$
 </p>
 <p className="text-slate-400 leading-rela
 Парадокс: один запрос за{" "}
 <code className="text-emerald-400"> $O(n)$ </code>
 всю структуру вдвое более плоской за сч
 при двойных вращениях {
 <code className="text-emerald-400 font-
 }. Дорогая операция инвестирует в дешев
 операций. Поэтому амортизированное врем
 <strong className="text-white"> $O(\log n)$
 </p>
 </div>
 <div className="mt-3 text-[10px] text-emera
 Каждое "растяжение" компенсируется "склад
 </div>
 </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 66/85: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИ

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';

```



```
// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESET
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху с

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('⚠ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }
}
```

```

 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 };

const topIndex = dirtyStack.length - 1;

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-3xl">Правило</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попробуешь вытащить нижнюю — всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 rounded-95 transition-all cursor-pointer flex items-center justify-center">
 Смыть (POP)
 </button>
 </div>
 </div>
);

```

```

 <div className="absolute w-5 h-5 rounded-1
"></div>
 <div className="absolute text-4xl anim-cr
</div>
)}

{dirtyStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {dirtyStack.slice().reverse().map(({plate,
 const idx = dirtyStack.length - 1 - rev
 const isTop = idx === topIndex;
 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
 shadow-md select-none transition-all duration

```

```


 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </div>
) : (
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex
items-center justify-center gap-2 opacity-1
 >
 {plate.name}

 </div>
 </div>
)
 </div>
)}
</div>

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-blue-500 to-purple-500"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-700 p-4"
 <div className="text-[10px] font-mono text-slate-200"
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ?
 {entry}
 </div>
 </div>
)
 </div>
 </div>

```

```

 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 совпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])

```

```

 else return generateZSteps(safePattern, safeText
 }, [mode, safePattern, safeText]);

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte
const reset = () => { setAutoPlay(false); setStepId

const step = steps[stepIdx] || steps[0];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp

```

```

 if (step.highlight?.include
 borderColor = "border-a
 bgColor = step.match ==
-900/40");
 }
}

// Z Logic
if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-a
 bgColor = step.match ==
-900/40");
 }
}

return (
 <div key={index} className=
 {/* L/R markers for Z */}
 {mode === "z" && step.l <= index && index <= step.r}
 <div className="abs
 >>
 {mode === "z" && step.r <= index && index <= step.l}
 <div className="abs
 >>

 {/* Index */}
 <span className={`text-

 {/* Char Box */}
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

```

```

 {mode === "z" && step.z}
 <div className="abs
 <span className=
 </div>
 }}

 {/* Found flash effect
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 {step.found && mode ===
 <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
 }}
 </div>
)
 }}}
</div>
</div>

<div className="flex flex-col sm:flex-row g
 {/* Controls */}
 <div className="flex bg-slate-900 borde
 <button onClick={prevStep} disabled
rounded-lg disabled:opacity-30 disabled:hov
 <Undo strokeWidth={3} size={16}
 </button>
 <button onClick={playPause} classNam
x items-center justify-center min-w-[80px]"
 {autoplay ? <><Pause size={16}
 </button>
 <button onClick={nextStep} disabled
r:bg-slate-800 rounded-lg disabled:opacity-
 <SkipForward strokeWidth={3} si
 </button>
 <button onClick={reset} className="
border-slate-800">

```



```
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}`}

</pre>
</div>
</div>
);
}
```

---

ФАЙЛ 68/85: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";

interface TooltipProps {
 /** Содержимое подсказки (можно JSX). */
 content: React.ReactNode;
 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху */
 side?: "top" | "bottom";
 className?: string;
}
```

---

ФАЙЛ 69/85: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from "react";
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";
```

```
interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}
```

```
interface TEdge {
 u: number;
 v: number;
}
```

```
const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];
```

```
const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
```

```

 "Начало топологической сортировки. Используем кла
 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, () => []);
 dagEdges.forEach((e) => adj[e.u].push(e.v));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name})`,
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in topological order)
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} от ${u}`,
 u,
);
 }
 }
 };

 fullyProcessed.add(u);
 dfsStack.pop();
 topoList.push(u); // prepend behavior: we write in reverse order
 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'`,
 u,
);
};

```

```

 }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button

```

```

 {/* Edges */}
 {dagEdges.map({edge, idx} => {
 const u = dagNodes.find((n) => n.id === edge.u)
 const v = dagNodes.find((n) => n.id === edge.v)

 const isActive =
 (step.currentNode === edge.u || step.currentNode === edge.v) ||
 step.visited.has(edge.v);

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-arrow" : "dag-line"})`}
 className="transition-all duration-300"
 />
);
 })}
)}
}

```

```

{/* Nodes */}
{dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id
 const isVisited = step.visited.has(node.id)
 const isProcessed = step.fullyProcessed.has(node.id)

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 }
})}

```

```

 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
 </g>
);
}
</svg>
</div>

```

```

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 rounded-00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

```

```

<div className="bg-slate-900 border border-slate-800">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидается обработка узлов...

) : (

```

```

 </button>
 <button
 disabled={currentStepIdx >= steps.length}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-800 text-white"
 >
 Вперед
 </button>
 </div>
 </div>
 </div>
</div>
);
};

```

ФАЙЛ 70/85: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";

```

```
export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающ
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки – это ровно то, ч
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы – куча сама держит лучшую
 Component: HeapViz,
 },
 "segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.
 Component: SegmentTreeVisualizer,
```



```
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем",
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
},
"bellman-ford": {
 title: "Форд-Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
},
floyd: {
 title: "Флойд-Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
},
"johnson-algo": { title: "Алгоритм Джонсона", Component: () => (
 "mst-kruskal": { title: "Краскал и DSU", Component: KruskalSimul
 "mst-prima": { title: "Прим", Component: KruskalSimul
 "mst-boruvka": { title: "Борувка", Component: KruskalSimul
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () => (
 "string-z-func": { title: "Z-функция", Component: () => (
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
 0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60, depth: 0 },
 // Betka H -> E -> R -> S
 1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, depth: 1 },
 2: { id: 2, label: 'HE', char: 'E', x: 200, y: 260, depth: 2 },
 3: { id: 3, label: 'HER', char: 'R', x: 150, y: 360, depth: 3 },
 4: { id: 4, label: 'HERS', char: 'S', x: 100, y: 460, depth: 4 },
 // Betka H -> I -> S
 5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, depth: 2 },
 6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, depth: 3 },
 // Betka S -> H -> E
 7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, depth: 1 },
 8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, depth: 2 },
 9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, depth: 3 },
 // Leaf nodes are 2 (HE)
};
```

```
// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]:
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};
```

```
export const WaterfallAnimationWidget: React.FC = () =>
 // Переключатель режимов: 'training' (Полный BFS обхо
 const [activeMode, setActiveMode] = useState<'trainin

 // === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
```

```

const char = textToScan[currentIndex];
let curr = currentNode;
let logMsg = `[Символ '${char}']`;

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}).`;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
} else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Лист`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} не имеет

 рыжок по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}).`;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 }
 curr = nextNode;
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

```

```

{
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди. Если вершина
 просто не существует. fail[H] автоматически напечатает
 сообщение об ошибке.',
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на Depth 1. Родитель
 вершины #7 (S) - вершина #1 (H).',
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
},
{
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель вершины
 #2 (HE) - вершина #1 (H). Разведчик плывет в ROOT и ищет ветку по "E".
 Если ветка не найдена, то вершина #2 (HE) не существует.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель вершины #5 (HI) -
 вершина #1 (H). Разведчик плывет в ROOT и ищет ветку по "I".
 "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},

```

```

 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С',
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. Ро
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTra

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border
 {/* Шапка и переключатель режимов */}
 <div className="flex flex-wrap items-center justify
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white

```

```

<div className="grid grid-cols-1 lg:grid-cols-3" style={{width: 1000px}}
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x" style={{width: 1000px}}
 <div>
 <h5 className="text-indigo-300 font-bold" style={{width: 1000px}}>
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3" style={{width: 1000px}}>
 Показаны все шаги очереди BFS (начиная с шага
 ar 8 (SHE → HE), где наглядно видно
 </p>
 <div className="space-y-1.5 mb-6 overflow" style={{width: 1000px}}
 {TRAINING_STEPS_INFO.map((step, idx) =>
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 py-2
 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-medium'
 : 'bg-slate-800 text-slate-400'
 }`}
 >
 {step.description}
 {idx === trainStep && Текущий шаг
 </button>
)}
 </div>
 </div>

 <div className="flex gap-2" style={{width: 1000px}}>
 <button
 onClick={() => setTrainStep((prev) => Math.min(prev + 1, TRAINING_STEPS_INFO.length - 1))}
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg-slate-600 text-white font-medium"
 >
 Назад

```

```

 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
/span>
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item
-4 border-blue-500 text-blue-300 font-bold'
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold m
 <Lightbulb className="w-4 h-4 text-ambe
 </h5>
 <div className="bg-slate-800 p-4 rounded-:
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
</div>
) : (
/* ПАНЕЛЬ РЕЖИМА ПОИСКА */
<div className="grid grid-cols-1 lg:grid-cols-3
 <div className="bg-slate-900/60 p-5 rounded-x
 <div>

```

```

 }}
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium transition-colors
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 `}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 <ArrowRight className="w-4 h-4" />
 </button>
</div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 ⚡ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <div className={`p-1.5 rounded flex items-center border border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center border border-rose-500 text-rose-300 font-bold`} >
 2. while (curr !== root && !trie)
 {activeSearchCodeLine === 2 && прыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex items-center border border-rose-500 text-rose-300 font-bold`} >
 3. while (curr !== root && !trie)
 {activeSearchCodeLine === 3 && прыжок по fail!}
 </div>
 </div>
 </div>
</div>

```



```

<div className="flex items-center justify-between" style={{width: 1000px; height: 100px; background-color: #f0f0f0; border: 1px solid #ccc; padding: 10px; margin-bottom: 20px;">
 <h5 className="text-white font-bold text-base" style={{margin: 0; padding: 0; flex: 1; text-align: center; color: #334155; font-size: 1.2em; font-weight: bold;}}>
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs" style={{margin: 0; padding: 0; flex: 1; text-align: center; font-size: 0.8em; font-weight: normal;}}>

 </div>
 </div>

```

```

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar" style={{width: 1000px; height: 400px; background-color: #f0f0f0; border: 1px solid #ccc; margin-top: 20px; position: relative;}}>
 <svg viewBox="0 0 800 520" className="w-full h-full" style={{width: 100%; height: 100%;}}>
 <g>
 <!-- Горизонтальные линии и подписи уровней -->
 {[
 { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
 { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
 { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
 { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
 { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' },
]}.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />

```

```

 x={{fromNode.x + toNode.x} / 2}
 y={{fromNode.y + toNode.y} / 2 + 10}
 fill={isTrainingExamined ? (isMatchBranch ? 'red' : 'blue') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 >
 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В РЕЖИМЕ ОБУЧЕНИЯ */}
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
 (fromNode.y + toNode.y) / 2})`} >
 <rect x="-15" y="-12" width="30" height="24"
 strokeWidth="2" />
 <text x="0" y="5" fontSize="14" fontStyle="italic"
 {isMatchBranch ? ' ' : ' '}>
 {isMatchBranch ? ' ' : ' '}
 </text>
 </g>
)}
 </g>
 </g>
);
});
}}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады) */}
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.from];

 // В режиме обучения показываем fail-ссылки
 // Найдем индекс шага, на котором обработчик вызывается
 const stepIdxForNode = TRAINING_STEPS_INFOS.findIndex(
 (step) => step.nodeId === node.id
);
 if (activeMode === 'training' && trainStep === stepIdxForNode) {
 return (
 <g>
 <rect
 x={node.x} y={node.y} width={node.width} height={node.height}
 fill="white" stroke="red" strokeWidth="2" />
 <text
 x={node.x + node.width / 2} y={node.y + node.height / 2}
 text="fail"
 fontStyle="italic"
 fontColor="red"
 fontSize="12" />
 </g>
);
 }

 let isHighlighted = activeMode === 'search' && node.id === 0;
 let isFailed = activeMode === 'fail';

```

```

 { /* Внешнее свечение для активной верш
 {isCurrent && {
 <circle cx={node.x} cy={node.y} r="
 }}
 {isTargetChild && {
 <circle cx={node.x} cy={node.y} r="
 }}

 { /* Основной круг вершины */}
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24
 fill={isCurrent ? '#2563eb' : isTar
 stroke={isCurrent ? '#ffffff' : isTa
 strokeWidth={isCurrent || isTargetCh
 className="transition-all duration-
 />

 { /* Текст внутри вершины */}
 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ?
 fontSize={isCurrent || isTargetChild
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 { /* Метка искомой вершины в режиме об
 {isTargetChild && {
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70"
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>

```

```

 {activeMode === 'training' ? ' ♂ ' : ''}
 </text>
 </g>

 </svg>
</div>
</div>

{/* Список найденных совпадений (только в режиме поиска) */}
{activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded" style={{width: 100%;}}>
 <h5 className="text-white font-bold mb-3 text-sm">
 ♂ Улов лосося (Найденные слова)
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic">
 Пока ничего не поймали. Запустите шаги симуляции
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border border-emerald-800 text-white font-bold shadow animate-[scaleUp_0.3s_ease-out]"
 style={{width: 100%;}}
 >
 <CheckCircle2 className="w-4 h-4 text-emerald-400" />
 {match.word}

 Индекс {match.index}

 </div>
 </div>
 </div>
)
)}
</div>
)}
</div>
)}
</div>

```

```

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 return (
 <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto">
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 mode === key ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"
 }`}
 >
 {label}
 </button>
))}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v-1] + v],
 []
);
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент A[i]
const [hover, setHover] = useState<{ kind: "pref" | "elem"; i: number}>({ kind: "pref", i: 0 });
const [range, setRange] = useState<{ l: number; r: number}>({ l: 0, r: 0 });

```

```

 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 }` }
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
})))
</div>

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" && i === range.l;
 const isL = i === range.l;
 const isR = i === range.r + 1;
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: "pref" })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help ${
 active
 ? "bg-emerald-500 text-white ring-1 ring-emerald-500"
 : isR
 ? "bg-emerald-700 text-white"
 : isL
 ? "bg-rose-700 text-white"
 : "bg-slate-900 text-slate-300"
 }` }
 title={`P[${i}] = сума a[0..${i} - 1]`}
 >
 {v}
 </div>
);
 })}
 </div>
</div>

```

```

 </div>

 {/* Запрос */}
 <div className="bg-slate-900 border border-indigo-500" style={borderStyle} >
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву)
 </div>
 <p className="font-mono text-sm sm:text-base text-slate-300">
 $\text{sum}(a[\text{range.l}..\text{range.r}]) = P[\text{range.r} + 1]$
 {pref[range.r + 1]}
 {pref[range.l]}
 {a[range.l]}
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Один вычитание вместо цикла: запрос за $O(1)$ по префикс-сумме
 </p>
 </div>
 </div>
);
};

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array.from({ length: m + 1 }, () => 0));
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1];
 }
 }
 return p;
 }, [n, m]);

 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
 const [rect, setRect] = useState<{ r1: 1, c1: 1, r2: 1, c2: 1 }>({ r1: 1, c1: 1, r2: 1, c2: 1 });

```

```

 {v}
 </div>
);
 }}}
</div>
)))
</div>
</div>

{/* Матрица префиксных сумм */}
<div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 <div className="inline-block">
 {P.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const corner = cornerOf(i, j);
 const isHover = hover?.i === i && hover?.j === j;
 const cornerColor =
 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner === "C"
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border-slate-700";
 return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i, j })}
 onMouseLeave={() => setHover(null)}
 className={` ${cellBase} cursor-help ${
 isHover ? "bg-amber-500 text-white" : ""
 }`}
 >
 {v}
 {corner && !isHover && (
 <span className="absolute -top-4 -right-4"

```



```
};
```

ФАЙЛ 73/85: src/components/visualizers/SparseTableViz.t

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТР0К: 709 | РАЗМ

```
import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";
```

```
// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;
```

```
const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
 }
}
```

```
// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
```

```

 }
 }

export const SparseTableViz: React.FC = () => {
 const [tab, setTab] = useState<"1d" | "2d_build" | "2d_query">("1d")

 return (
 <div className="w-full bg-slate-950 p-3 sm:p-4 md:p-5">
 <div className="flex gap-2 sm:gap-4 mb-5 border-b border-slate-800">
 <button
 onClick={() => setTab("1d")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 1. Сворачивание 1D
 </button>
 <button
 onClick={() => setTab("2d_build")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-slate-400 text-white" : "bg-slate-800 text-slate-400`}
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
)
}
```

```

 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, st
 className="p-2 bg-indigo-600 hover:bg-indigo
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-
 >
 <RotateCcw size={20} />
 </button>
</div>
</div>

<div className="bg-slate-900 rounded-xl border bo
 <div className="text-[11px] uppercase tracking-t
 <div className="flex gap-1 sm:gap-1.5 overflow-
 {arrlD.map((v, idx) => {
 const inCover = !!covered && idx >= covered
 return {
 <div key={idx} className="flex flex-col i
 <div
 className={`flex items-center justify
 w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
 inCover ? "bg-indigo-500 text-white
 }`}
 >
 {v}
 </div>
 <span className="text-[9px] text-slate-
 </div>
);
 })}
)}

```

```

</div>
<div className="text-center font-bold text-slate-500">
 2^2 (L=4)
</div>
<div className="text-center font-bold text-slate-500">
 2^3 (L=8)
</div>

{/* Matrix */}
{Array.from({ length: N }).map((_, i) => {
 <React.Fragment key={i}>
 <div className="text-slate-500 font-mono">
 {i}
 </div>
 {Array.from({ length: LOG }).map((_, j) => {
 const isValid = i + (1 << j) <= N;
 const isVisible =
 isValid &&
 (j === 0 ||
 computeSteps.findIndex((s) => s.i === i + (1 << j) && s.j === j + 1) <= 0);

 // Active computing state
 let isActive = false;
 let isSrc1 = false;
 let isSrc2 = false;

 if (step > 0 && step <= maxStep) {
 const currentStep = computeSteps[step];
 if (currentStep.i === i && currentStep.j === j) {
 isActive = true;
 }
 if (currentStep.j === j + 1 && currentStep.i === i) {
 isSrc1 = true;
 }
 if (
 currentStep.j === j + 1 &&
 currentStep.i === i - (1 << (currentStep.j - j))
) {
 isSrc2 = true;
 }
 }

 return (
 <div className="text-slate-500 font-mono">
 {i + (1 << j)}
 </div>
);
 })}
 </React.Fragment>
})}

```

```

 }` }
 >
 {stlD[i][j]}
 </motion.div>
))

 {/* Arrows visualization */}
 {(isSrc1 || isSrc2) && (
 <motion.svg
 initial={{ opacity: 0 }}
 animate={{ opacity: 1 }}
 className="absolute pointer-e
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <<
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc
 fill="none"
 stroke={isSrc1 ? "#10b981"
 strokeWidth="3"
 />
 </motion.svg>
))
 </div>
);
}})
</React.Fragment>
)))
</div>
</div>
</div>

{/* Formula helper text */}

```

```
};
```

```
const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number }>({ r: 0, c: 0 });

 const L = 1 << k;
 const activeCell = locked || hover;

 return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scrollable">
 <h3 className="text-xl font-bold text-indigo-400">Визуализация
 <p className="text-sm text-slate-400 mb-6 text-wrap">
 Для наглядности показываем только <b className="text-rose-300">

 иц!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null); setLocked(null); }}
 className={`px-4 py-2 rounded-md text-sm text-slate-400 hover:text-white hover:bg-slate-900`}
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${step + 1}`}
 </button>
))}
 </div>

 <div className="flex flex-col items-center gap-6 overflow-y-scroll no-scrollbar">
 {[...Array(k + 1)].map((_, i) => k - i).map((step, i) => {
 const stepL = 1 << step;
```

```

 if (isTop && isLeft) {
 highlightClass = 'bg-rose
md:scale-[1.08]';
 } else if (isTop && !isLeft) {
 highlightClass = 'bg-blue
] md:scale-[1.08]';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emer
9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amber
3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';
 }
}

return (
 <div
 key={idx}
 onMouseEnter={() => { if(is
 onMouseLeave={() => { if(is
 onClick={() => {
 if(isCurr) {
 if(locked && locked.
 else setLocked({r, c
 }
 }}
 className={`flex items-centr
sCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp
clamp(20px,5.2vw,30px)] text-[clamp(8px,2vw
>
 {ST2D[r][c][step][step]}
 </div>
);
}}}
</div>
</div>

```

```

span>

 c + pL][k-// Правый Низ

 {'}'}));
</div>

{activeCell ? (
 <div className="bg-slate-950 p-5 rounded-2xl mt-2 transition-all">
 <p className="text-slate-300 mb-3">
 Пример для ячейки:
 {locked && <Lock size={
 </p>
 <p className="text-indigo-300 border">

 }}[k]][k]

 </p>

 <div className="space-y-2 pt-3 pl-2">
 {(() => {
 const prevL = 1 << (k - 1);
 return (
 <div className="flex items-center">
 </div> ST[{activeCell.r}[activeCell.c][k-1][k-1]}
 </div>
 <div className="flex items-center">
 </div> ST[{activeCell.r}[activeCell.c + prevL][k-1][k-1]}
 </div>
)
 })}
 </div>
)}
)

```



```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);

 const H = r2 - r1 + 1;
 const W = c2 - c1 + 1;
 const kx = Math.floor(Math.log2(H));
 const ky = Math.floor(Math.log2(W));
 const Lx = 1 << kx;
 const Ly = 1 << ky;

 const matRows = 8 - Lx + 1;
 const matCols = 8 - Ly + 1;

 return (
 <div className="flex flex-col xl:flex-row gap-6"
 <div className="flex-1 min-w-0 bg-slate-900"
 <h3 className="text-xl font-bold text-rose-500">
 <p className="text-slate-400 text-[13px] font-normal">
 еpx, r2, c2 - правый низ)</p>

 <div className="flex flex-col items-center"
 <div className="grid grid-cols-[repeat(8, 1fr)] gap-2"
 <div className="border border-rose-500/50 border-scale-105 z-10"
 <div key={idx} className={`text-[clamp(9px,2.4vw,12px)]
 y-center rounded text-[clamp(9px,2.4vw,12px)]
 0 border-rose-500/50 border scale-105 z-10"
 {val2D[r][c]}
 </div>
 </div>
 </div>
 </div>
 </h3>
 </div>
 </div>
)
)
}

```

```

14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
 </div>
 </div>
</div>

```

```

<div className="flex-1 bg-slate-900 p-6 rounded
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.
 </p>
 <p className="text-sm font-sans text-sla
 Чтобы покрыть весь прямоугольник, мы
Name="font-mono bg-[#0a0fle] px-1 rounded b
роса. (Цветные прямоугольники слева).
 </p>

```

```

 <div className="bg-[#0a0fle] rounded-xl
text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3
 // 1.
 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<

```

```

const c = idx % matCols;
let isTL = r === r1 && c === 0;
let isTR = r === r1 && c === matCols - 1;
let isBL = r === r2 - 1 && c === 0;
let isBR = r === r2 - 1 && c === matCols - 1;

let color = "bg-slate-800 text-white";
let txt = ST2D[r][c][kx][ky];

let badge = null;
if (isTL) { color = "bg-rose-400 border-2"; badge = "TL"; }
else if (isTR) { color = "bg-blue-300 border-2"; badge = "TR"; }
else if (isBL) { color = "bg-emerald-300 border-2"; badge = "BL"; }
else if (isBR) { color = "bg-amber-300 border-2"; badge = "BR"; }

return (
 <div key={idx} className={color}
 transition-all duration-300 relative shrink-0 ${c === 0 ? "w-100px" : "w-100px"}
 {txt}
 {badge && <div className="text-white px-1 font-bold rounded" />}
 </div>
)
)}}}
</div>
</div>
</div>

<div className="mt-auto bg-slate-950 font-sans text-white" >
 <div className="text-center font-bold text-white" style={
 wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border-2 border-white
 }>


```

```

 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

const aside = document.querySelector('aside');
const headerMain = document.querySelector('header main');
const questionsContainer = document.getElementById('questions');
const wtfContainer = document.getElementById('wtf');

if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname);
 window.scrollTo(0, 0);
} else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
}

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]').forEach(btn => {
 btn.classList.remove('outline', 'outline-1');
 });

 if (theme === 'dark') document.getElementById('dark-mode').checked = true;
 if (theme === 'light') document.getElementById('light-mode').checked = true;
}

```

```

 drawerOverlay.classList.add("open");
 setTimeout(() => drawerOverlay.classList.remove("open"), 300);
 }
 document.body.style.overflow = "";
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => {
 mobileDrawer.toggleDrawer();
 });
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => {
 mobileDrawer.toggleDrawer();
 });
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => {
 mobileDrawer.toggleDrawer();
 });
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains("open")) {
 mobileDrawer.toggleDrawer();
 }
 });
});

const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
};

const observer = new IntersectionObserver((entries) => {
 entries.forEach(entry => {
 if (entry.isIntersecting && !document.querySelector("#mobile-drawer a.active")) {
 // Highlight active TOC link location
 const id = entry.target.id;
 document.querySelectorAll('aside a').forEach(link => {
 link.classList.remove("active");
 if (link.id === id) {
 link.classList.add("active");
 }
 });
 }
 });
});

```

```

 }
 };
</script>
<style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "../src/index.css";
 .math-font { font-family: 'Fira Code', monospace; }
 .uno-card {
 width: 70px; height: 105px; border-radius: 10px;
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); position: relative;
 align-items: center; justify-content: center; color: white; overflow: hidden; flex-shrink: 0;
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #000; font-family: 'Arial Black', Impact, sans-serif;
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute; text-shadow: 1px 1px 0 #000, -1px -1px 0 #000;
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; transform: rotate(180deg); }
 .uno-card.mini { width: 45px !important; height: 60px !important; }
 .uno-card.mini::before, .uno-card.mini::after {
 font-size: 0.8em;
 }

 /* Global formula scroll settings */
 .formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
 }
 .formula-scroll::-webkit-scrollbar {
 display: none;
 }

 mjx-container {
 max-width: 100% !important;
 }

```

```

 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card
 .card-green { background-color: #339e35; } .card
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-ligh
body.theme-light .bg-slate-800 { background-col
body.theme-light .bg-slate-700, body.theme-ligh
 important; }
body.theme-light .text-slate-200, body.theme-li
body.theme-light .text-slate-400 { color: #47556
body.theme-light .border-slate-600, body.theme-
body.theme-light .border-slate-500 { border-col
body.theme-light #top-controls { background-col

body.theme-notebook {
 background-color: #fcf8eb !important; /* li
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient
 background-attachment: local !important;

```

```

 </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light"
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook"
 transition-colors">Заметки</button>
 </div>
 </div>
 </div>

 <!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
 <svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" markerHeight="6"
 viewBox="0 0 6 6" fill="yellow">
 <path d="M0,0 L6,3 L0,6 Z" fill="yellow" />
 </marker>
 <marker id="arrow-orange" markerWidth="6" markerHeight="6"
 viewBox="0 0 6 6" fill="orange">
 <path d="M0,0 L6,3 L0,6 Z" fill="orange" />
 </marker>
 <marker id="arrow-pink" markerWidth="6" markerHeight="6"
 viewBox="0 0 6 6" fill="pink">
 <path d="M0,0 L6,3 L0,6 Z" fill="pink" />
 </marker>
 </defs>
 </svg>

```



```

 <!-- Ручка -->
 <path d="M 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333" />
 <!-- Выход -->
 <path d="M 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#fff" />
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="10" />
 <rect x="3" y="5" width="14" height="5" />
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20" />
 <rect x="6" y="-5" width="8" height="6" />
 <rect x="26" y="-5" width="8" height="6" />
 <line x1="0" y1="2" x2="40" y2="2" stroke="black" />
 <text x="20" y="14" fill="#475569" font-weight="bold">
 ЛЕГО
 </text>
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) --&;
<button id="mobile-menu-btn" class="lg:hidden fixed top-0/50 z-50 hover:bg-blue-500 transition-transform lg:w-10 lg:h-10"
 <svg xmlns="http://www.w3.org/2000/svg" class="lg:w-10 lg:h-10"
 <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" />
 </svg>
</button>

<!-- Затемнение фона на мобилках при открытом меню
<div id="drawer-overlay" class="lg:hidden fixed inset-0 bg-black bg-opacity-50 transition-opacity duration-300"
 </div>

<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-direction-column"
 </div>

 <!-- Левое меню (Оглавление) -->
 <aside id="mobile-drawer" class="fixed inset-y-0 left-0 w-full transition-transform duration-300 ease-in-out lg:block lg:w-72 lg:shrink-0 lg:z-auto"

```

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-teal-500 pl-3 font-bold text-teal-300">

 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text-teal-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">

 4. Степени и корни

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-amber-500 pl-3 font-bold text-amber-300">

 5. Формулы

 </summary>
 <div class="pl-6 space-y-2 text-amber-300">

 </div>
</details>

```

```
</summary>
<div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class=
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
 <a href="#q8-5" class="bloc
 <a href="#q8-6" class="bloc
 <a href="#q8-7" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-9" class=
er-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q9-1" class="bloc
 <a href="#q9-2" class="bloc
```

```
</details>

<details class="group">
 <summary class="list-none cursor-pointer-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые многочлены)

</summary>
<div class="pl-6 space-y-2 text-blue-400">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

</summary>
<div class="pl-6 space-y-2 text-emerald-400">

</div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer-fuchsia-500 pl-3 font-bold text-fuchsia-400">
 14. Кратность корня.

</summary>
```

```

 <div class="mt-4 mb-2 text-xs font-l
 <a href="#question-17" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-tea

 <a href="#question-18" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-blur

 <a href="#question-19" class="block
ald-500 pl-3 text-slate-400">
 <span class="font-bold text-eme

 <a href="#question-20" class="block
-500 pl-3 text-slate-400">
 <span class="font-bold text-ros

 <a href="#question-21" class="block
go-500 pl-3 text-slate-400">
 <span class="font-bold text-ind

 <a href="#question-22" class="block
sia-500 pl-3 text-slate-400">
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="
-red-500 pl-3 mt-4 text-red-500 font-bold m
 ▲ Ликбез: Алгоритмы выживания

</nav>
<script>
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA

```

```

 // Open the parent
 const corresponding
 if (correspondingLi
 const details =
 if (details &&
 details.open
 document.qu
 if (othe
 othe
 }
 });
 }
}
});
}, {
 root: null,
 rootMargin: '-10% 0px -70%',
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

function fallbackModal(contentRaw,
 const modal = document.createEl
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = '
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'co
 modal.style.alignItems = 'cente

```

```

copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#fff';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';
closeBtn.style.cursor = 'pointer';
closeBtn.onclick = () => modal.close();

box.appendChild(header);
box.appendChild(subHeader);
if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(copyBtn);
}
btnContainer.appendChild(closeBtn);
box.appendChild(btnContainer);
modal.appendChild(box);
document.body.appendChild(modal);

```

```

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark')?.remove();
 document.getElementById('theme-light')?.remove();
 document.getElementById('theme-style')?.remove();

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark')?.innerHTML = `
 body.bg-slate-900 section h1 { color: white; }
 body.bg-slate-900 main { color: white; }
 body.bg-slate-900 main p { color: white; }
 /* Universal text color */
 body.bg-slate-900 .text-slate-900 { color: white; }
 body.bg-slate-900 .text-slate-900 { color: white; }
 body.bg-slate-900 .text-slate-900 { color: white; }
 body.bg-slate-900 [class*="text-slate-900"] { color: white; }

 /* Specific backgrounds */
 body.bg-slate-900 aside > div { color: white; }
 body.bg-slate-900 [class*="bg-slate-900"] { color: white; }
 `;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light')?.innerHTML = `
 body.bg-white section h1 { color: black; }
 body.bg-white main { color: black; }
 body.bg-white main p { color: black; }
 /* Universal text color */
 body.bg-white .text-slate-900 { color: black; }
 body.bg-white .text-slate-900 { color: black; }
 body.bg-white .text-slate-900 { color: black; }
 body.bg-white [class*="text-slate-900"] { color: black; }

 /* Specific backgrounds */
 body.bg-white aside > div { color: black; }
 body.bg-white [class*="bg-slate-900"] { color: black; }
 `;
 }

 style.innerHTML = `
 body.bg-slate-900 section h1 { color: white; }
 body.bg-slate-900 main { color: white; }
 body.bg-slate-900 main p { color: white; }
 /* Universal text color */
 body.bg-slate-900 .text-slate-900 { color: white; }
 body.bg-slate-900 .text-slate-900 { color: white; }
 body.bg-slate-900 .text-slate-900 { color: white; }
 body.bg-slate-900 [class*="text-slate-900"] { color: white; }

 /* Specific backgrounds */
 body.bg-slate-900 aside > div { color: white; }
 body.bg-slate-900 [class*="bg-slate-900"] { color: white; }
 `;

 style.innerHTML += `
 body.bg-white section h1 { color: black; }
 body.bg-white main { color: black; }
 body.bg-white main p { color: black; }
 /* Universal text color */
 body.bg-white .text-slate-900 { color: black; }
 body.bg-white .text-slate-900 { color: black; }
 body.bg-white .text-slate-900 { color: black; }
 body.bg-white [class*="text-slate-900"] { color: black; }

 /* Specific backgrounds */
 body.bg-white aside > div { color: black; }
 body.bg-white [class*="bg-slate-900"] { color: black; }
 `;
}

```



```

 `;
 } else if (theme === 'terminal')
 body.classList.add('bg-black')
 document.getElementById('theme')
 .style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: transparent !important;
 }
 body.bg-black section h1 {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace !important;
 }
 body.bg-black svg path,
 stroke: #22c55e !important;
 fill: transparent !important;
 }
 `;
 }

 document.head.appendChild(style)
}
</script>

```

```

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 border-1
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-500 font-
 <button onclick="downloadWord()"
ont-bold flex items-center justify-center gap-2
 Word
 </button>
 <a href="algebra.pdf" target="

```

4">

## ВЫСШАЯ АЛГЕБРА

</h1>

<p class="text-slate-400 italic text-lg">

<!-- Print Table of Contents -->

<div class="hidden print:block mt-8 text-center">  
<h2 class="font-bold text-xl mb-4">

<ul class="text-base space-y-2">

<li><a href="#question-1">1. Алгебра

<li><a href="#question-2">2. Комбинаторика

<li><a href="#question-3">3. Теория вероятностей

<li><a href="#question-4">4. Статистика

<li><a href="#question-5">5. Группы

<li><a href="#question-6">6. Декартовы координаты

<li><a href="#question-7">7. Алгебра

<li><a href="#question-8">8. Кривые

<li><a href="#question-9">9. Координаты

<li><a href="#question-10">10. Алгебра

<li><a href="#question-11">11. Комбинаторика

<li><a href="#question-12">12. Теория вероятностей

<li><a href="#question-13">13. Статистика

<li><a href="#question-14">14. Группы

<li><a href="#question-15">15. Декартовы координаты

<li><a href="#question-17">17. Алгебра

<li><a href="#question-18">18. Кривые

<li><a href="#question-19">19. Координаты

<li><a href="#question-20">20. Алгебра

<li><a href="#question-21">21. Комбинаторика

<li><a href="#question-22">22. Теория вероятностей

</ul>

</div>

</header>

<!-- ===== WTF CONTAINER =====>

<div id="wtf-container" class="hidden">

<h2 class="text-3xl font-black text-center">

```


 <a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 3: НОД и НОК / Алгоритм Ев
 <a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-12" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структур
 <a href="#question-8" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate


```

```

 <div class="font-bold text-slate-700">

- l-4 border-blue-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-blue-500">
 <div class="font-bold text-slate-700">

- l-4 border-emerald-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-emerald-500">
 <div class="font-bold text-slate-700">

- l-4 border-rose-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-rose-500">
 <div class="font-bold text-slate-700">

- l-4 border-indigo-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-indigo-500">
 <div class="font-bold text-slate-700">

- l-4 border-fuchsia-500 hover:bg-slate-700 transition-colors
 <div class="text-xs text-fuchsia-500">
 <div class="font-bold text-slate-700">

 <!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ -->
 <div class="col-span-1 md:col-span-3" style="border: 1px solid red; padding: 10px; margin-top: 10px;">
border-b border-red-500/30 pb-2">Кластер 4:

:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4 border-t-4 border-b-4
 <div class="text-xs text-red-500">
 <div class="font-bold text-slate-700">


```

```

<!-- От противного -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-fuchsia-400">От противного
 <p class="text-sm text-slate-400">Предположим, что
 <ul class="list-disc pl-5" style="list-style-type: none;">
 и

 т.д.).

 что так и задумано: "Ой, противоречие, теорема

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-emerald-400">Единственность
 <p class="text-sm text-slate-400">Пусть
 <ul class="list-disc pl-5" style="list-style-type: none;">

 , r_2).

 счет ограничений (размер остатка) докажи, что

</div>

<!-- Индукция -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-yellow-400">Индукция
 <p class="text-sm text-slate-400">Пусть
 <ul class="list-disc pl-5" style="list-style-type: none;">

 валось.

 Просто перепиши формулу с буквой k .

 усок k , замени его на жареного карася из

```

```
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
ициентов (они делятся на p), кроме старшего
зложить.</p>
```

```
</div>
```

```
 <div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-slate-400">
. Чаще всего после этого Фейс-контроль p .
```

```
</div>
```

```
</div>
```

```
 <h2 class="text-2xl font-bold text-slate-400">
(Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
<ul class="space-y-4 pb-10">
```

```
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400">
 <p class="text-sm text-slate-400">
 <code class="text-xs font-slate-400">
 </div>
```

```

```

```
 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan-400">
 <p class="text-sm text-slate-400">
```

```
х двух, потом последних.

```

Универсальное пособие • полный исходный код

---

ФАЙЛ 76/85: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 77/85: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 228 | РАЗМЕР: 9.8 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";

/**
 * Выгрузка главы (или всего пособия) в самостоятельный
 *
 * Файл получается автономным: стили зашиты внутрь, инт
 * его можно открыть с флешки, отправить в мессенджер и
 * Живые React-визуализации в статику не переносятся —
 * подставляется заметка со ссылкой на онлайн-версию, а
 * которые в приложении всплывают по наведению, собираю
 * в конце документа.
 */

/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
```

```

pre { white-space: pre-wrap; }
};

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string): { body: string; termIds: string[] } {
 const host = document.createElement("div");
 host.innerHTML = html;

 try {
 annotateTerms(host);
 } catch {
 /* если браузер не осилил регулярку – выгружаем текст как есть */
 }

 const ids: string[] = [];
 host.querySelectorAll<HTMLElement>(".term-hint").forEach((el) => {
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разбегается
 host.querySelectorAll("details").forEach((d) => d.setOpen());

 return { body: host.innerHTML, termIds: ids };
}

```



```

<header class="export-head">
 <div style="font-size:.7rem;text-transform:uppercase">
 <h1 style="color:#fff;font-size:1.6rem;font-weight:bold">
 <div style="font-size:.72rem;color:#475569;margin-top:10px">
 </div>
</header>

<div class="export-note">
 Это автономная копия: стили внутри файла, интернет
 Интерактивные тренажёры и анимации в статичный HTML
 они живут в онлайн-версии: <a href="${escapeHtml(opts.op
</div>

<article class="prose prose-invert max-w-none">
${opts.body}
</article>

${opts.gloss}

<footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}

```

```

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {

```

```
const body = `
```

---

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

---

-----  
ФАЙЛ 78/85: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

-----

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
```

```

 });
 });

const has2D = (r) => r.interactive || r.inlineSvg || r.svg;
const gapBoth = rows.filter((r) => !r.hasAnalogy && !has2D(r));
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && has2D(r));
const gapViz = rows.filter((r) => r.hasAnalogy && !has2D(r));
const orphanViz = [...vizIds].filter((id) => !chapters[id]);

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) => i + 1).filter(n => !nums.includes(n));

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |");
 console.log("| --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy ? "да" : "нет"} | ${r.svg ? "да" : "нет"} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(`  * ${r.title} [${r.num}]`));
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missingNums.join(", ")}`);
 console.log(`Визуализаторы без главы: ${orphanViz.join(", ")}`);
}

```

```

console.log("тем | подсв. | глава");
for (const c of chapters) {
 const text = c.content
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
 .replace(/<[>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
);
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");
console.log("Не встретились:", GLOSSARY.filter((g) => !bad.includes(g)).length, "шт.");

```

```
 Subject: "Экспорт всего кода проекта (код -> txt
 CreationDate: new Date(),
 },
));

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise(({resolve, reject}) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`      страниц: ${pages.length}`);
console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

---

ФАЙЛ 81/85: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 1.2 КБ

---

```
/**
 * ШАГ 1: КОД -> TXT
 *
```

```
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
}
```

```

const out = [];
const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { ti
const totalBytes = files.reduce((s, f) => s + fs.stat

push(HR);
push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода: ${humanSize(totalByt
push(`Глав/билетов в пособии:        ${chapters.length}`)
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
push(HR);
chapters.forEach((c, i) => {
 push(` ${String(i + 1).padStart(3, " ")} ${c.titl
});
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")} ${rel}`)
});

```

```
main();
```

---

ФАЙЛ 82/85: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ: 1.5 КБ

---

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_all_pages.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_all_pages.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
};
```



ФАЙЛ 83/85: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМЕР: 3.5 КБ

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT,
 PAGES_DIR,
 TXT_PATH,
 PAGE,
 ensureDir,
 humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/ /g, "");
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min((expanded.match(/^ /g).length || 0), PAGE.cols));
 let rest = expanded;
 let first = true;
 while (rest.length > 0) {
 const width = first ? PAGE.cols : PAGE.cols - indent.length;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
}
```

```

 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "Helvetica",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

```

```

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

```

```

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}

```

```

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

const missing = jobs.filter((j) => !fs.existsSync(j.pngFile));
if (missing.length) throw new Error(`Не отрендерены ${missing.length} страниц`);

```

```

const bytes = jobs.reduce((s, j) => s + fs.statSync(j.pngFile).size, 0);
console.log("[2/3] txt -> png");
console.log(`объем: ${bytes} байт, страниц: ${total}`);

```

```
contents: read
pages: write
id-token: write
```

```
concurrency:
 group: pages
 cancel-in-progress: true
```

```
jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist
```

Универсальное пособие • полный исходный код

```

└───────────> page-0001.png ... page-NNNN.png (
#
└──────────> full_code_all_page
#
Готовые TXT и PDF коммитятся обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.

on:
 push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бескон
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умол
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}`
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[
 steps:
 - name: Checkout
```

- name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
- name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| \`public/export/full\_code\_all\_pages`"  
 echo "| PNG-страниц | `\${ls tmp/export-pages}`"  
 } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt  
 if-no-files-found: error  
 retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
 name: full-code-pages-png  
 path: tmp/export-pages/\*.png  
 if-no-files-found: error  
 retention-days: 7
- name: Commit rebuilt export back to the branch  
run: |  
 git config user.name "github-actions[bot]"  
 git config user.email "41898282+github-action"